INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SÃO PAULO
Câmpus São Carlos

TTEC - Tópicos Tec. Emergentes

Introdução a Linguagem RUST p.2

Conteúdo 02

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SÃO PAULO
Câmpus São Carlos

Lembrando umas coisas da aula passada...

Máquina virtual

- Ubuntu, mais ambiente gráfico Lxde

- “Capada”, não tem quase nada, pra ficar leve
  - Terminal
  - Editor de textos simples (porém gráfico)
  - Navegador web velho...
  - **Compilador RUST !**

- Usuário **`wecomp`**
  - senha: **`wecomp`**

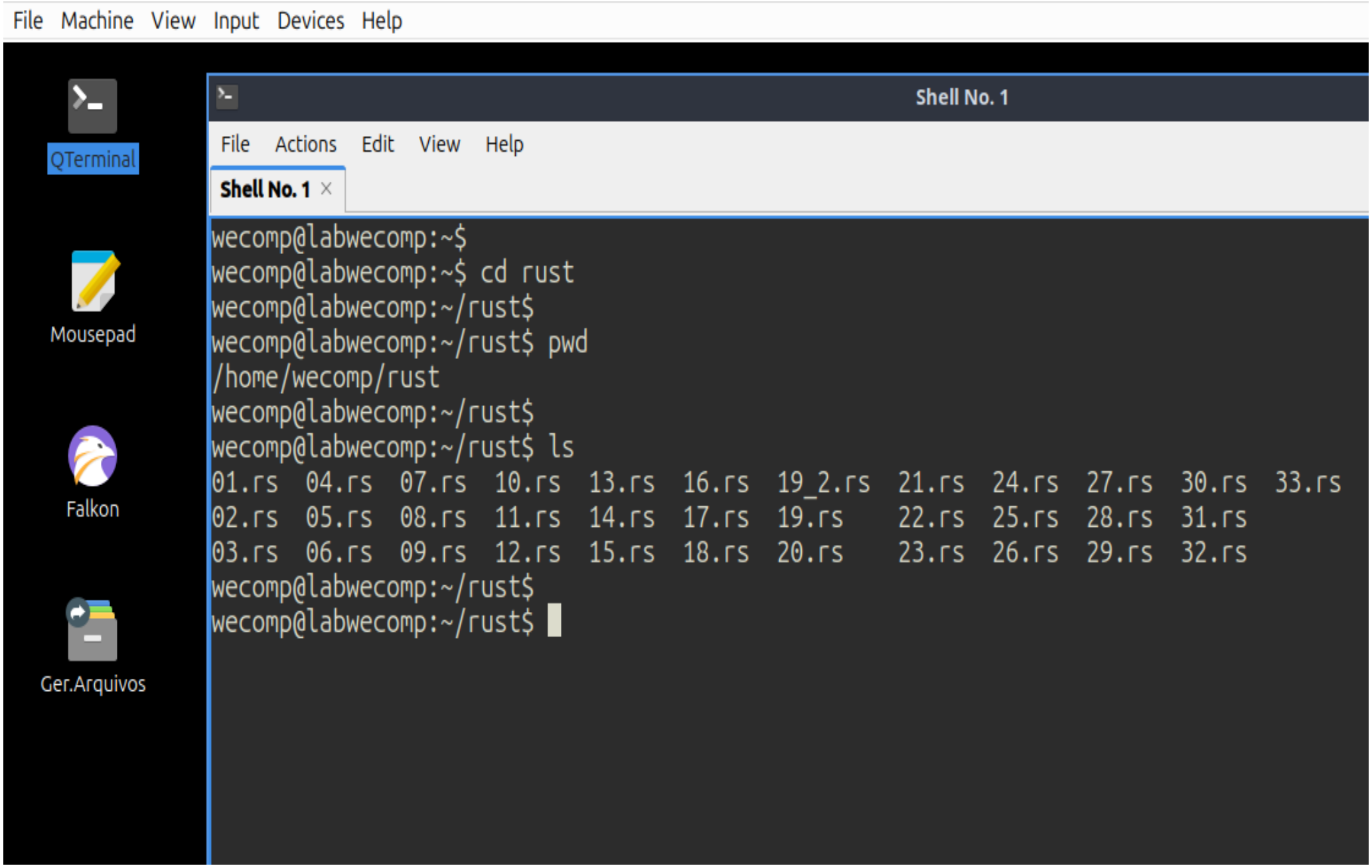
Conteúdo disponível em

https://drive.google.com/drive/folders/1sP011P7y5tM1GmVZf1nx-p1Uc4Iw4e6t?usp=drive_link

Não vamos usar nenhuma IDE.
A idéia aqui é mostrar uma introdução.
Vamos usar a linha de comando (terminal).

No Linux, a pasta do usuário fica em
`/home/wecomp`

Os códigos-fonte estão numa subpasta,
`/home/wecomp/rust`


File Machine View Input Devices Help
QTerminal
Mousepad
Falkon
Ger.Arquivos
Shell No. 1
File Actions Edit View Help
Shell No. 1
wecomp@labwecomp:~$
wecomp@labwecomp:~$ cd rust
wecomp@labwecomp:~/rust$
wecomp@labwecomp:~/rust$ pwd
/home/wecomp/rust
wecomp@labwecomp:~/rust$
wecomp@labwecomp:~/rust$ ls
01.rs 04.rs 07.rs 10.rs 13.rs 16.rs 19_2.rs 21.rs 24.rs 27.rs 30.rs 33.rs
02.rs 05.rs 08.rs 11.rs 14.rs 17.rs 19.rs 22.rs 25.rs 28.rs 31.rs
03.rs 06.rs 09.rs 12.rs 15.rs 18.rs 20.rs 23.rs 26.rs 29.rs 32.rs
wecomp@labwecomp:~/rust$
wecomp@labwecomp:~/rust$

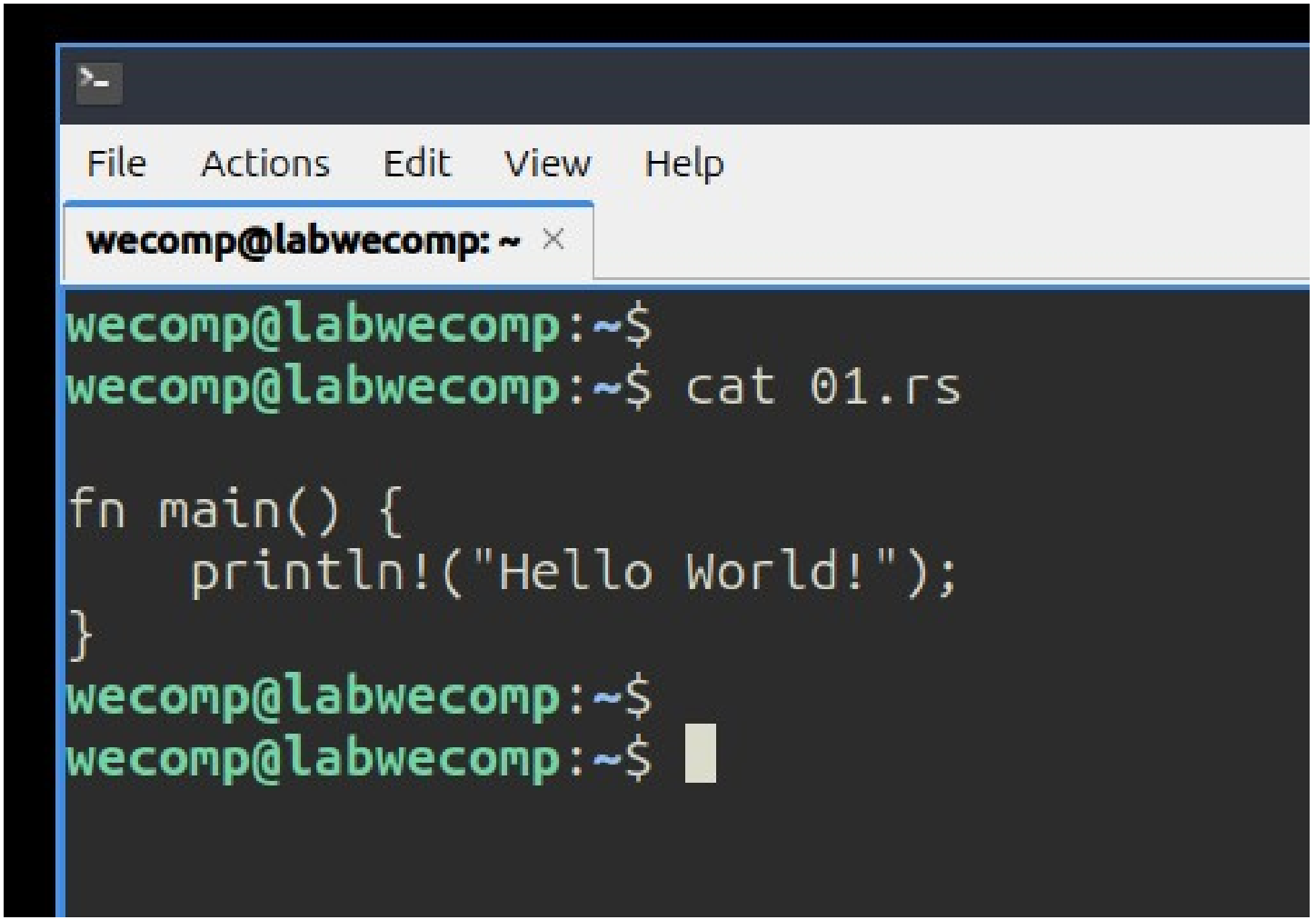
Nos slides a seguir,
os comandos foram dados
a partir do diretório home do usuário
(**~**).

Para realizar a recompilação
precisa estar dentro da pasta rust,
como mostrada.

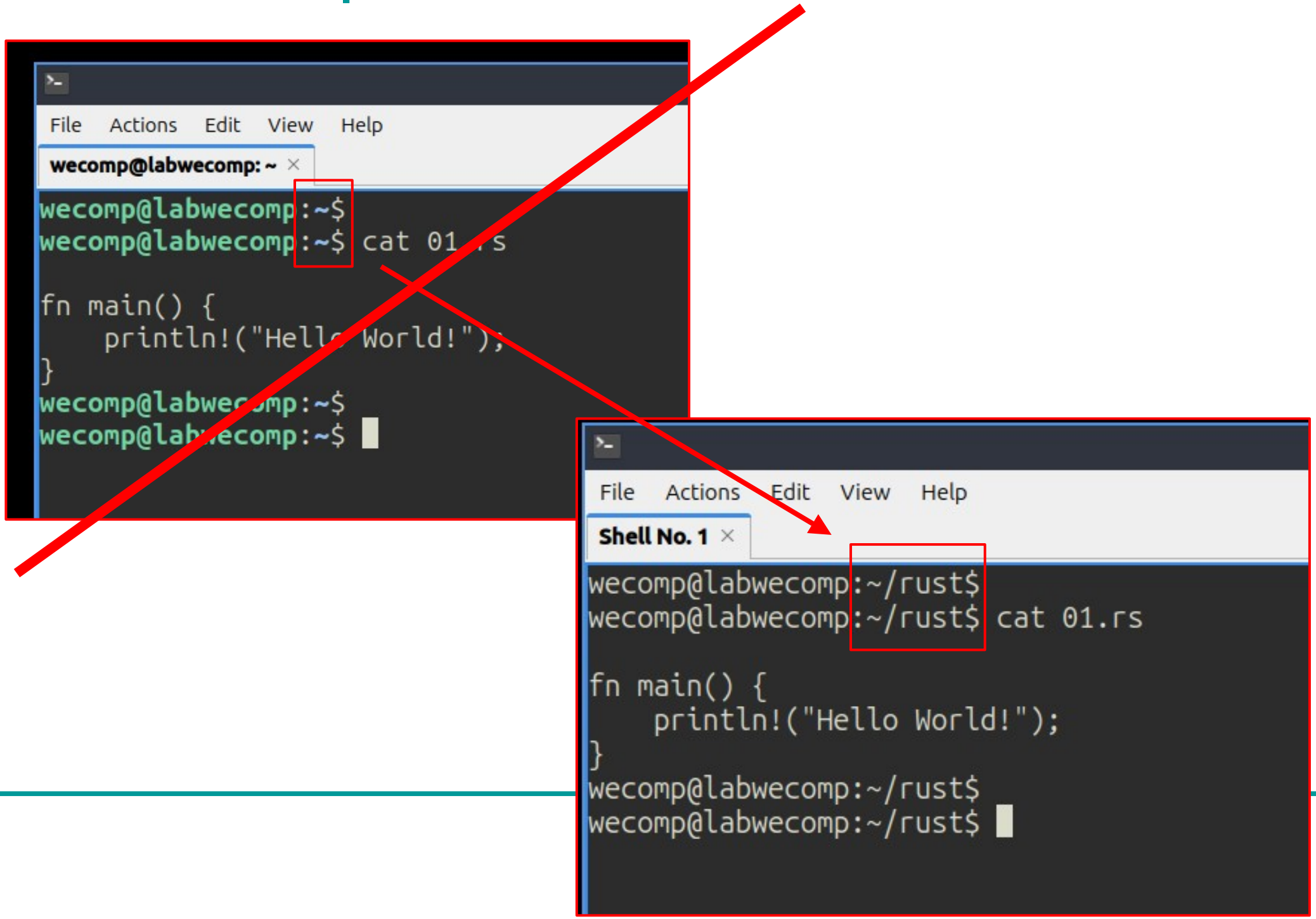
Algumas dicas para manipulação
dos arquivos de códigos...

Lembrando,
usaremos apenas o terminal de comandos.

Visualizar conteúdo de arquivo no terminal: cat



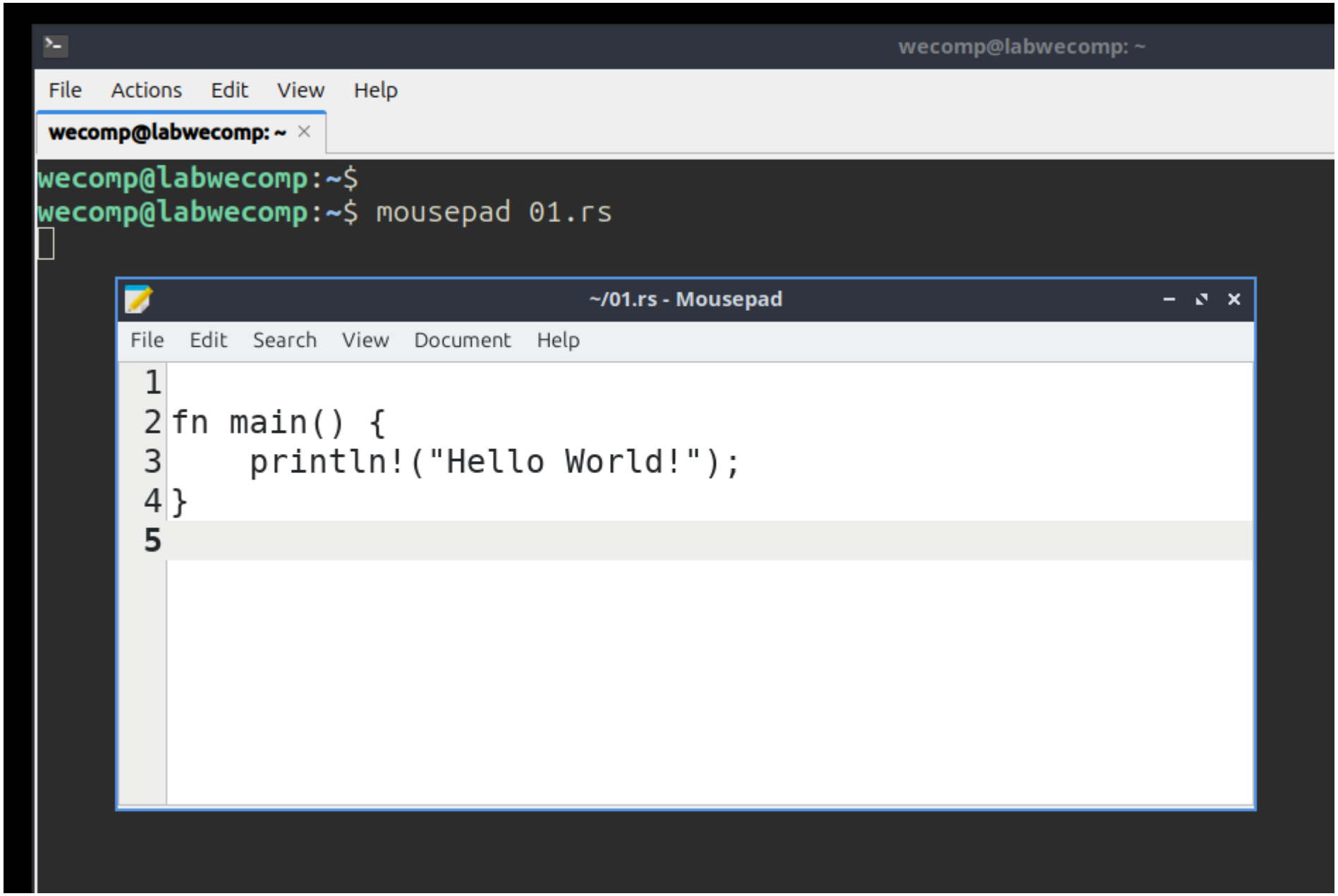
Lembrando.... pasta rust !!!!



Visualizar conteúdo de arquivo com editor “mousepad”

Pode abrir o editor, file|open, blablabla...

Ou abrir direto do terminal o arquivo desejado.



Variáveis

- Criação com **`let`**
- Tipos:
  - i8, i16, ...
  - u8, u16, ...
  - f32, f64
  - bool, char, &str, ()

Tipos de variáveis

🔢 Tipos inteiros

Inteiros podem ser **com sinal** (aceitam negativos e positivos) ou **sem sinal** (apenas positivos).

Com sinal (signed)

- `i8` → 8 bits → intervalo de **-128 a 127**
- `i16` → 16 bits → intervalo de **-32.768 a 32.767**
- `i32` → 32 bits → intervalo de **-2.147.483.648 a 2.147.483.647**
- `i64` → 64 bits → intervalo de **-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807**
- `i128` → 128 bits → intervalo muito maior, raramente usado.
- `isize` → tamanho depende da arquitetura:
  - 32 bits em sistemas 32-bit
  - 64 bits em sistemas 64-bit

Sem sinal (unsigned)

- `u8` → 8 bits → **0 a 255**
- `u16` → 16 bits → **0 a 65.535**
- `u32` → 32 bits → **0 a 4.294.967.295**
- `u64` → 64 bits → **0 a 18 quintilhões e pouco**
- `u128` → 128 bits → ainda maior.
- `usize` → mesmo tamanho que `isize`, mas só aceita valores positivos (muito usado para índices em

🌊 Tipos de ponto flutuante

- `f32` → 32 bits (precisão simples, similar ao `float` em C/Java).
- `f64` → 64 bits (precisão dupla, similar ao `double`).

> Por padrão, Rust usa `f64` quando você escreve algo como `let x = 3.14;`.

imutabilidade

- Por padrão, variáveis em RUST são imutáveis.
- Não confunda com constante
- A variável pode ter seu valor atribuído na primeira vez, mas depois não pode mais ter seu valor alterado.
- Se quiser poder mudar o valor atribuído, precisa usar a palavra chave “mut” na definição da variável.

Variáveis locais, globais, static, unsafe

Resumo – `let`, `const` e `static` em Rust

- `let`
  - Cria variáveis locais (escopo limitado).
  - Imutável por padrão, pode ser `mut`.
  - Vive apenas enquanto o escopo existir.
- `const`
  - Valor fixo, substituído em tempo de compilação.
  - Pode ser declarado no escopo global ou local.
  - Sempre imutável e sem endereço em memória.
- `static`
  - Variável global única, alocada em memória fixa.
  - Vive durante toda a execução do programa.
  - Imutável por padrão; `static mut` existe, mas exige `unsafe`.

Conceito de escopo

Em Rust, **escopo** é a região do código onde uma variável, constante, função ou recurso é válido e pode ser acessado. Ele começa no ponto onde o item é declarado e termina quando o bloco em que foi definido se encerra.

De forma prática:

- Uma variável declarada dentro de um bloco `{ ... }` só pode ser usada dentro desse bloco e deixa de existir quando o bloco termina.
- Esse comportamento está diretamente ligado ao sistema de **ownership** e **lifetime** de Rust: quando o escopo acaba, os valores alocados são liberados automaticamente.

Independentemente de qualquer coisa, chaves ({ ... }) definem um bloco.

shadowing

Shadowing em Rust

- **Shadowing** é o ato de declarar uma nova variável com o mesmo nome de uma já existente.
- A nova variável **sombra** a anterior dentro do mesmo escopo ou em um escopo mais interno.
- A variável original não desaparece: ela apenas fica inacessível enquanto a nova estiver ativa.
- Quando o escopo da variável mais recente termina, a versão anterior volta a ser visível (se ainda existir).
- Esse recurso permite reutilizar nomes de forma organizada, evitando conflitos e mantendo o código claro.

Funções

Mas antes...
tudo em RUST é uma expressão

(e o que o ; tem a ver com isso!!)

Expressão vs. Declaração em Rust

- **Expressão**: algo que *produz um valor*.
  Exemplos:
  - `2 + 3` → resulta em `5`
  - `x * 10`
  - `{ let y = 5; y + 1 }` → resulta em `6`
- **Declaração (statement)**: algo que *não produz valor diretamente*, mas sim faz uma ação (ex.: definir uma variável).
  Exemplo:

```rust
let a = 10; // declaração de variável
```

  Mas observe: `10` é uma **expressão** dentro dessa declaração.

Papel do ponto e vírgula (`;`)

- Quando você **não coloca** `;` no final de uma linha, significa que aquela expressão deve **retornar um valor**.
- Quando você **coloca** `;`, a expressão se transforma em uma declaração que não retorna valor (o tipo vira `()`, chamado de *unit type* em Rust).*

👉 Resumindo:

- **Tudo em Rust é expressão** (até blocos `{ ... }`).
- O `;` **decide** se você quer usar o valor da expressão ou apenas executar e descartar, resultando em `()`.

Expressões e `;` em Rust

- **Tudo é expressão** → produz um valor
- **Blocos `{ ... }` também são expressões**
- O `;` muda o comportamento:

📌 **Sem `;`** → a expressão retorna um valor

📌 **Com `;`** → o valor é descartado, o resultado é `()` (*unit type*)

O **unit type** em Rust é escrito como `()`.

Ele representa:

- **A ausência de um valor significativo** (semelhante ao `void` em C/Java, mas é de fato um tipo).
- **Um único valor possível**, que é o próprio `()` — por isso é chamado *unit* (unidade).

10.rs

```
 1
 2 fn soma( a:i32, b:i32 ) -> i32
 3 {
 4     println!( "{} + {} = {}", a, b, a + b );
 5     a + b  // sem o ; é uma expressão, está retornando este valor
 6 }
 7
 8 fn main() {
 9
10     println!( "Soma = {}", soma(3, 7) );
11
12 }
13
```

```rust
fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

}
```

1. **Definição da função** `soma`

```rust
fn soma(a: i32, b: i32) -> i32
```

- A função recebe dois parâmetros `a` e `b` do tipo `i32`.
- A seta `-> i32` indica que a função deve **retornar um valor inteiro.**

```rust
fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

}
```

2. **Corpo da função**

rust — Copiar código

```rust
println!("{} + {} = {}", a, b, a + b);
a + b
```

- Primeiro imprime na tela a soma.
  Exemplo: `3 + 7 = 10`.
- Depois, a última linha `a + b` **é uma expressão sem** `;`, logo o valor calculado é **retornado** pela função.

> Se tivesse `a + b;` (com ponto e vírgula), isso não retornaria nada, e o compilador reclamaria porque a função prometeu devolver um `i32`.

```rust

fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

}

```

3. **Função** `main`

```rust
println!("Soma = {}", soma(3, 7));
```

- Chama `soma(3, 7)`.
- A função imprime:

```
3 + 7 = 10
```

- Depois retorna `10`.
- Esse valor é usado no `println!`, resultando em:

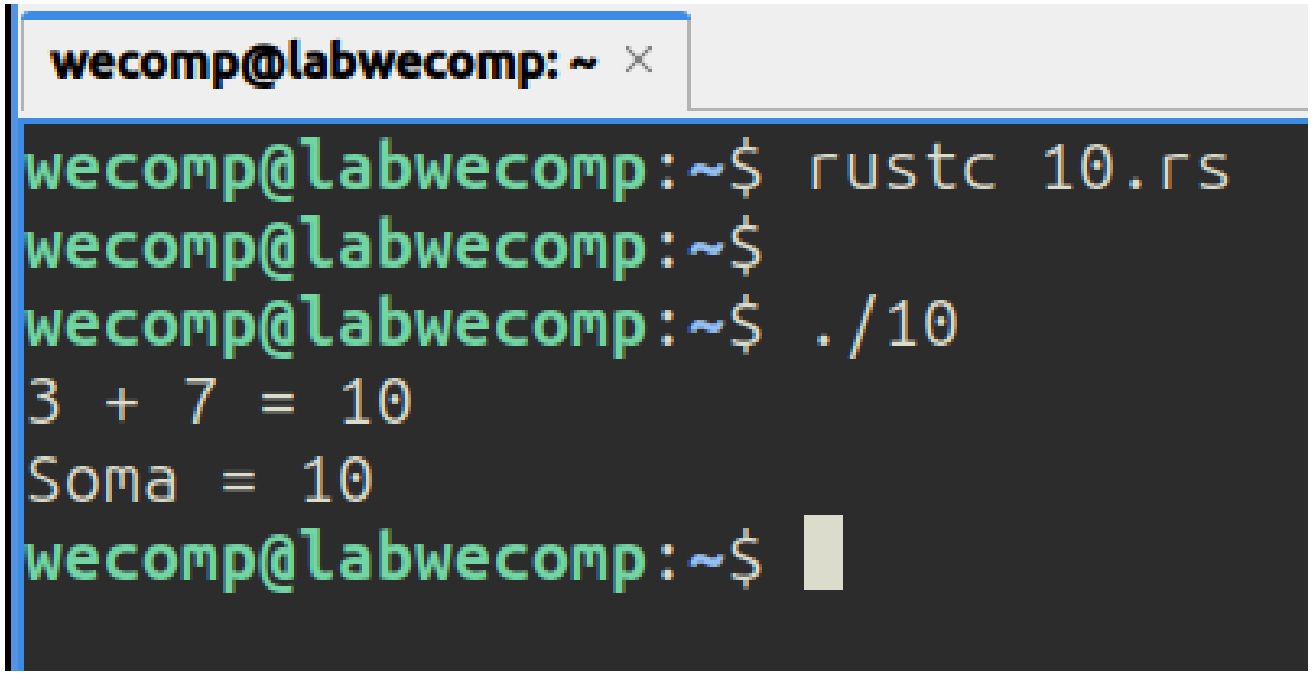
```ini
Soma = 10
```

```rust
fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

}
```



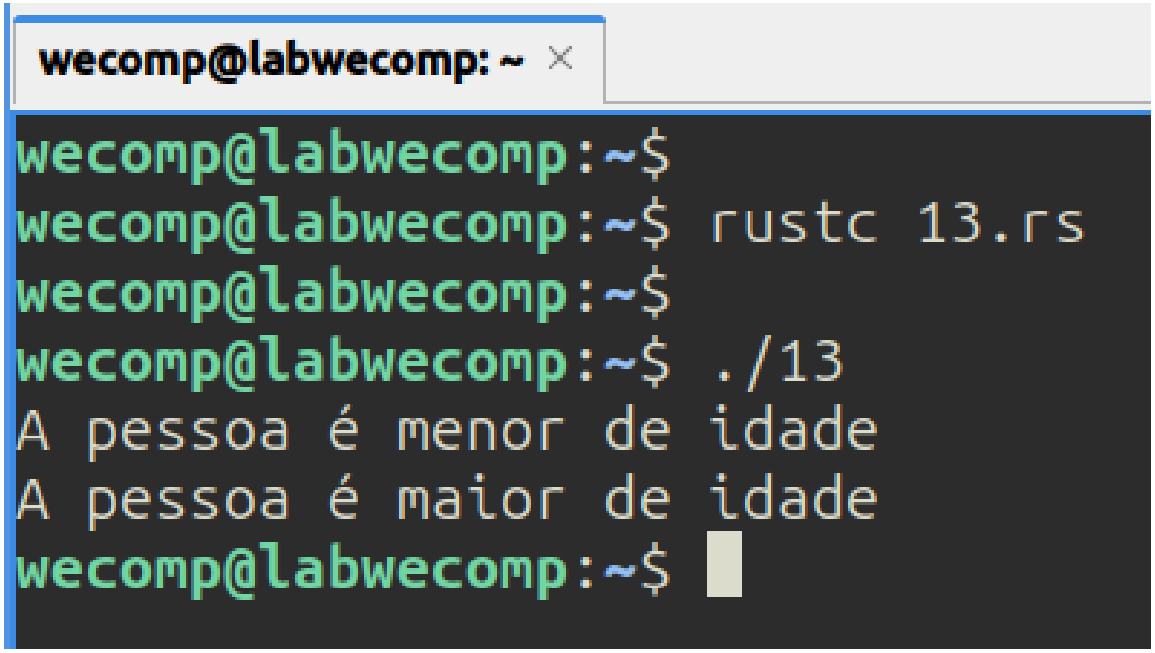
Condição

Lembrando que em RUST tudo pode ser uma expressão (retorna um valor), basta não usar o ;

13.rs

```
1
2 fn main() {
3
4     let mut idade = 16;
5
6     let mut maioridade = if idade > 17 { "maior" } else { "menor" };
7
8     println!("A pessoa é {} de idade", maioridade);
9
0     idade = 20;
1     maioridade = if idade > 17 { "maior" } else { "menor" };
2     println!("A pessoa é {} de idade", maioridade);
3
4 }
5
```

```
1
2 fn main() {
3
4     let mut idade = 16;
5
6     let mut maioridade = if idade > 17 { "maior" } else { "menor" };
7
8     println!("A pessoa é {} de idade", maioridade);
9
0     idade = 20;
1     maioridade = if idade > 17 { "maior" } else { "menor" };
2     println!("A pessoa é {} de idade", maioridade);
3
4 }
5
```



Continuando...

Laços



14.rs

```
fn main() {

    let multiplicador:u8 = 5;

    let mut contador:u8 = 1;

    while contador <= 10 {

        println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);

        contador += 1;

    }

}

```

14.rs

```rust
fn main() {

    let multiplicador:u8 = 5;

    let mut contador:u8 = 1;

    while contador <= 10 {

        println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);

        contador += 1;

    }

}
```

```
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 14.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./14
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
wecomp@labwecomp:~$
```

Uma consideração sobre “parênteses” (vale pro if também)

```
 2 fn main() {
 3
 4     let multiplicador:u8 = 5;
 5
 6     let mut contador:u8 = 1;
 7
 8     while ( contador <= 10 ) {
 9
10         println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);
11
12         contador += 1;
13
14     }
15
16 }
17
```

```rust
fn main() {

    let multiplicador:u8 = 5;

    let mut contador:u8 = 1;

    while ( contador <= 10 ) {

        println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);

        contador += 1;

    }

}
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 14.rs
warning: unnecessary parentheses around `while` condition
 --> 14.rs:8:11
  |
8 |     while ( contador <= 10 ) {
  |           ^^               ^^
  |
  = note: `#[warn(unused_parens)]` on by default
help: remove these parentheses
  |
8 -     while ( contador <= 10 ) {
8 +     while contador <= 10 {
  |

warning: 1 warning emitted
```

```
2 fn main() {
3
4     let multiplicador:u8 = 5;
5
6     let mut contador:u8 = 1;
7
8     while ( contador <= 10 ) {
9
10        println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);
11
12        contador += 1;
13
14    }
15
16 }
17
```

```
warning: 1 warning emitted

wecomp@labwecomp:~$ ./14
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
wecomp@labwecomp:~$
```

“Como faz do...while ?”

“Como faz do...while ?”



Dá pra simular com loop ... break

No **Rust**, o comando `loop` cria um laço **infinito**.

Ele vai executar o bloco de código indefinidamente até que você **interrompa** com `break` ou **pule** para a próxima iteração com `continue`.

15.rs

```
fn main() {

    let multiplicador:u8 = 5;

    let mut contador:u8 = 0;

    loop {

        contador += 1;

        println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);

        if contador == 10 {
            break;
        }
    }
}
```

15.rs

```rust
fn main() {

    let multiplicador:u8 = 5;

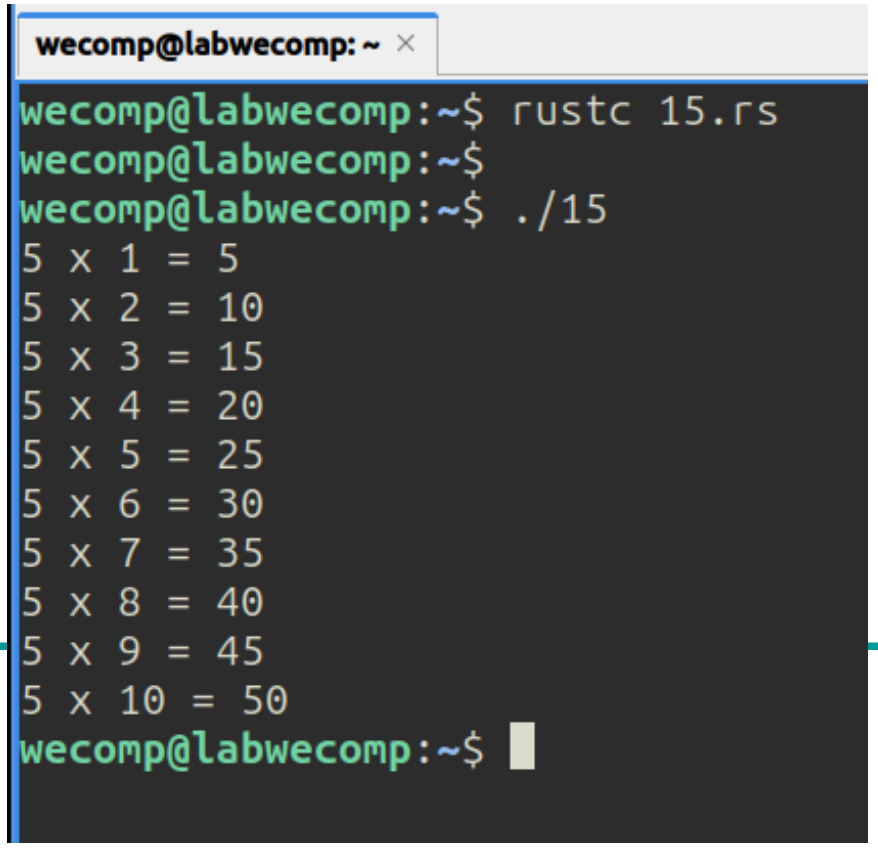
    let mut contador:u8 = 0;

    loop {

        contador += 1;

        println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);

        if contador == 10 {
            break;
        }
    }
}
```



Um detalhe sobre isso...

```
fn main() {

    let multiplicador:u8 = 5;

    let mut contador:u8 = 0;

    loop {

        contador += 1;

        println!("{} x {} = {}", multiplicador, contador, multiplicador*contador);

        if contador == 10 {
            break;
        }
    }
}
```

```
if contador == 10 {
    break;
}
```

Em **Rust**, assim como em C, C++ e Java, se o `if` (ou `else`) tem **apenas uma instrução**, as chaves `{}` podem ser omitidas.

No seu exemplo, o trecho:

rust Copiar código

```rust
if contador == 10 {
    break;
}
```

poderia ser escrito assim:

rust Copiar código

```rust
if contador == 10
    break;
```

e o programa funcionaria da mesma forma.

⚠️ Porém, na prática, é **muito comum usar sempre as chaves em Rust**, mesmo com uma linha só. Isso aumenta a clareza e evita erros em futuras alterações (o compilador até aceita sem chaves, mas o estilo oficial do **rustfmt** sempre adiciona `{ }`).

```rust
if contador == 10 {
    break;
}
```

Em **Rust**, assim como em C, C++ e Java, se o `if` (ou `else`) tem **apenas uma instrução**, as chaves `{}` podem ser omitidas.

No seu exemplo, o trecho:

rust — Copiar código

```rust
if contador ==
    break;
}
```

poderia ser escrito a

rust

```rust
if contador ==
    break;
```

e o programa funcionaria da mesma forma.

⚠️ Porém, na prática, é **muito comum usar sempre as chaves em Rust**, mesmo com uma linha só. Isso aumenta a clareza e evita erros em futuras alterações (o compilador até aceita sem chaves, mas o estilo oficial do **rustfmt** sempre adiciona `{ }`).

O `rustfmt` é a ferramenta oficial de formatação automática de código em Rust.

- Ele lê seu código-fonte e reorganiza de acordo com o **estilo padrão da comunidade**.
- Assim, todos os projetos Rust seguem uma aparência uniforme, facilitando leitura e manutenção.
- É similar ao `gofmt` no Go ou ao `prettier` no JavaScript.

break / continue

`break`

- Encerra a execução imediata do laço (`loop` , `while` ou `for`).
- Em Rust, além de simplesmente sair do laço, pode **retornar um valor** (o que não existe em C/Java).

Exemplo:

rust

Copiar código

```rust
fn main() {
    let mut n = 0;
    let resultado = loop {
        n += 1;
        if n == 5 {
            break n * 2; // sai do loop e retorna 10
        }
    };
    println!("resultado = {}", resultado);
}
```

`continue`

- Pula o restante do bloco do laço e vai direto para a **próxima iteração**, como em C/Java.

```rust
fn main() {
    let mut i = 0;

    loop {
        i += 1;

        if i > 5 {
            break; // encerra o loop quando i passa de 5
        }

        if i % 2 == 0 {
            continue; // pula os números pares
        }

        println!("ímpar: {}", i);
    }
}
```

Saída

```makefile
ímpar: 1
ímpar: 3
ímpar: 5
```

🔎 O que acontece:

1. O contador `i` começa em 0.
2. A cada iteração, ele é incrementado.
3. Se `i` for par, o `continue` pula o `println!` e volta ao início do laço.
4. Se `i` for maior que 5, o `break` encerra o laço.

laço com for

O comando `for` em Rust

- O `for` em Rust serve para **repetir um bloco de código percorrendo uma sequência de valores.**
- Diferente de C/Java, não existe o formato clássico `for (inicialização; condição; incremento)`.
- Em Rust, o `for` sempre percorre uma **faixa de números** ou uma **coleção de dados.**

16.rs

```rust
fn main() {

    // Não inclui o 6 (quem sabe python, entendeu!)

    for i in 1..6 {

        println!( "i = {}", i );

    }

    println!("----");

    // Se quiser incluir o 6:

    for i in 1..=6 {

        println!( "i = {}", i );

    }

}
```

16.rs

```rust
fn main() {

    // Não inclui o 6 (quem sabe python, entendeu!)

    for i in 1..6 {

        println!( "i = {}", i );

    }

    println!("----");

    // Se quiser incluir o 6:

    for i in 1..=6 {

        println!( "i = {}", i

    }

}
```

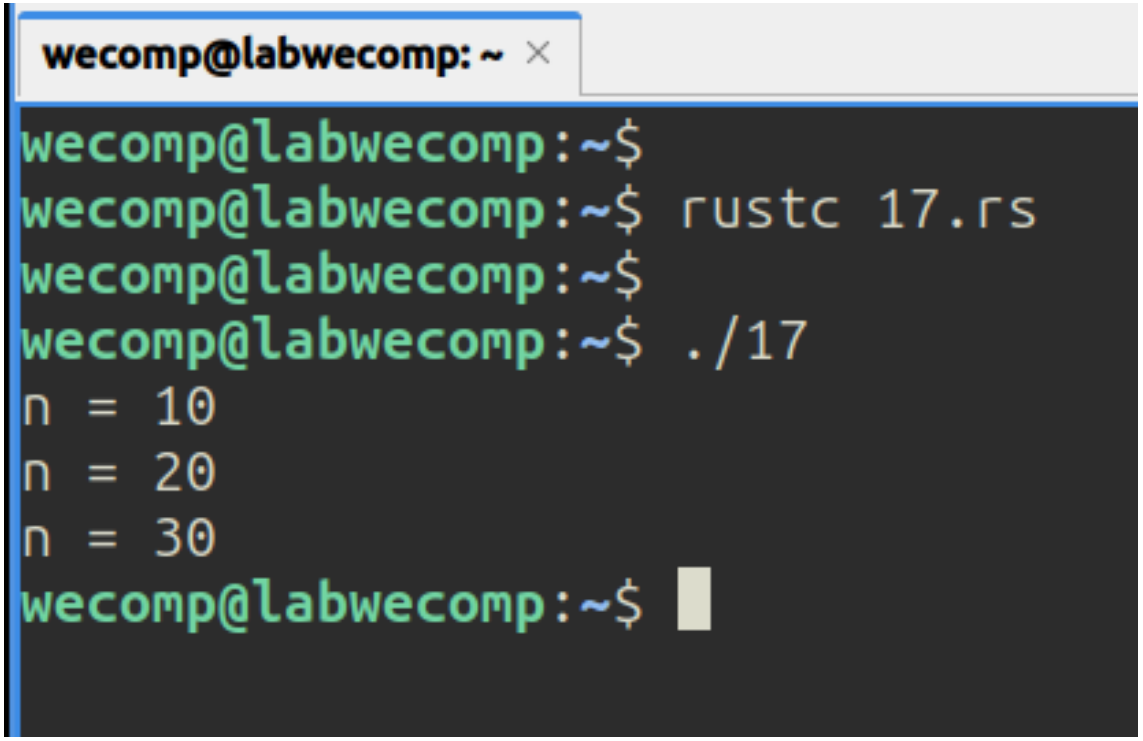
```
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 16.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./16
i = 1
i = 2
i = 3
i = 4
i = 5
----
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
wecomp@labwecomp:~$
```

17.rs

```
 1
 2 fn main() {
 3
 4     let numeros = [ 10, 20, 30 ];
 5
 6     for n in numeros {
 7
 8         println!( "n = {}", n );
 9
10     }
11
12 }
```

17.rs

```
 1
 2 fn main() {
 3
 4     let numeros = [ 10, 20, 30 ];
 5
 6     for n in numeros {
 7
 8         println!( "n = {}", n );
 9
10     }
11
12 }
```



match

O comando `match` em Rust

- `match` é usado para **comparar um valor com vários padrões possíveis**.
- É parecido com o `switch` de C/Java, mas **mais poderoso e seguro**, porque Rust exige que todos os casos possíveis sejam tratados.

18.rs

```
1
2 fn main() {
3
4     let     linguagem = "PHP";
5
6     let proposito = match linguagem {
7
8         "PHP" => "Web",
9         "Kotlin" => "Android",
10        "Python" => "Data Science"
11    };
12
13    println!("O proposito de {} eh {}", linguagem, proposito);
14
15 }
16
```

18.rs

```rust

fn main() {

    let     linguagem = "PHP";

    let proposito = match linguagem {

        "PHP" => "Web",
        "Kotlin" => "Android",
        "Python" => "Data Science"
    };

    println!("O proposito de {} eh {}", linguagem, proposito);

}

```

O que ele deveria fazer

O objetivo é simples: dado o valor da variável `linguagem`, o `match` escolhe o propósito correspondente e imprime, por exemplo:

mathematica — Copiar código

```
O proposito de PHP eh Web
```

18.rs

```rust
fn main() {

    let     linguagem = "PHP";

    let proposito = match linguagem {

        "PHP" => "Web",
        "Kotlin" => "Android",
        "Python" => "Data Science"
    };

    println!("O proposito de {} eh {}", linguagem, proposito);

}
```

```
wecomp@labwecomp:~$ rustc 18.rs
error[E0004]: non-exhaustive patterns: `&_` not covered
 --> 18.rs:6:27
  |
6 |     let proposito = match linguagem {
  |                           ^^^^^^^^^ pattern `&_` not covered
  |
  = note: the matched value is of type `&str`
  = note: `&str` cannot be matched exhaustively, so a wildcard `_` is necessary
help: ensure that all possible cases are being handled by adding a match arm with a wildcard
 pattern or an explicit pattern as shown
  |
10~         "Python" => "Data Science",
11+         &_ => todo!()
  |

error: aborting due to 1 previous error
```

18.rs

```rust
fn main() {

    let     linguagem = "PHP";

    let proposito = match linguagem {

        "PHP" => "Web",
        "Kotlin" => "Android",
        "Python" => "Data Science"
    };

    println!("O proposito de {} eh {}", linguagem, proposito);

}
```

Por que dá erro

O erro acontece porque **o `match` em Rust precisa tratar todos os casos possíveis.**

No seu código:

rust — Copiar código

```rust
let proposito = match linguagem {
    "PHP" => "Web",
    "Kotlin" => "Android",
    "Python" => "Data Science"
};
```

- `linguagem` é do tipo `&str`.
- `match` cobre apenas `"PHP"`, `"Kotlin"` e `"Python"`.
- **Se `linguagem` fosse outro valor**, o `match` não teria um padrão correspondente.
- Rust não permite isso — precisa de um **padrão "catch-all"** usando `_`.

Consertando...

19.rs

```
fn main() {

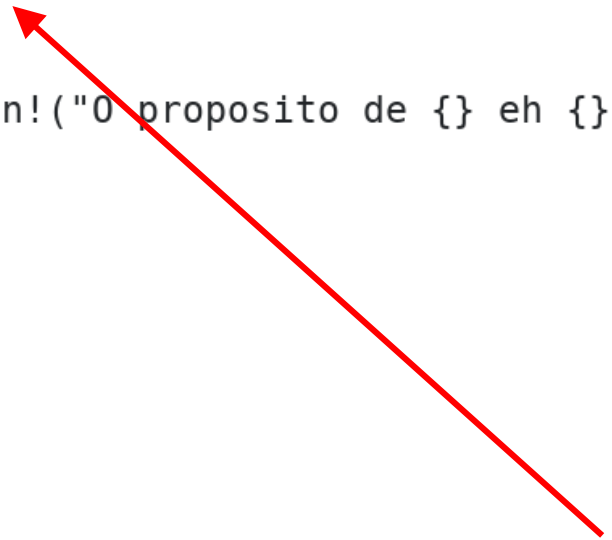
    let linguagem = "PHP";

    let proposito = match linguagem {

        "PHP" => "Web",
        "Kotlin" => "Android",
        "Python" => "Data Science",
        _ => "Desconhecido"
    };

    println!("O proposito de {} eh {}", linguagem, proposito);

}
```



19.rs

```rust
fn main() {

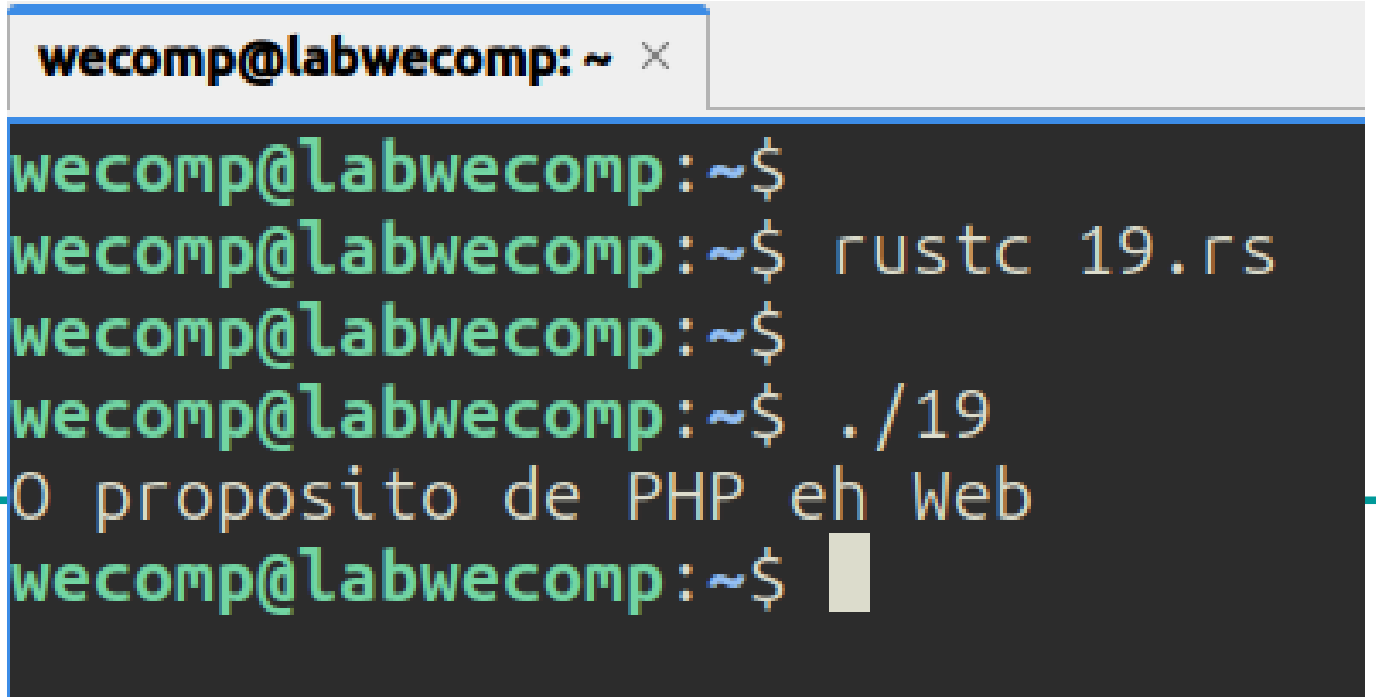
    let linguagem = "PHP";

    let proposito = match linguagem {

        "PHP" => "Web",
        "Kotlin" => "Android",
        "Python" => "Data Science",
        _ => "Desconhecido"
    };

    println!("O proposito de {} eh {}", linguagem, proposito);

}
```



```rust
fn main() {

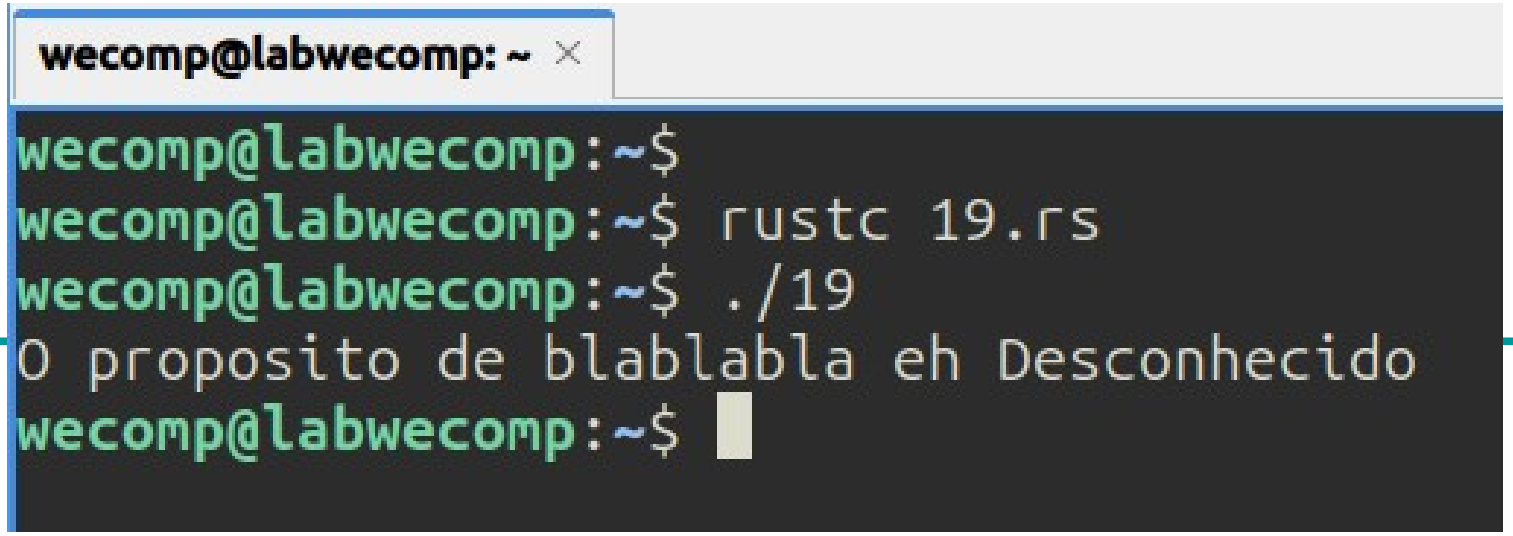
    let linguagem = "blablabla";

    let proposito = match linguagem {

        "PHP" => "Web",
        "Kotlin" => "Android",
        "Python" => "Data Science",
        _ => "Desconhecido"
    };

    println!("O proposito de {} eh {}", linguagem, proposito);

}
```



Mais um pouco de pattern matching...

19_2.rs

```
1
2  fn main() {
3
4      // gerar valores para x, de 1 até 20 (inclusive):
5
6      for x in 1..=20 {
7
8          // Vamos exibir x e uma mensagem relacionada ao valor de x:
9
10         println!( "{}: {}", x, match x {
11
12             // x = 1, exiba pouco:
13             1 => "Pouco",
14
15             // se x for 2 ou 3, exiba um pouquinho:
16             2 | 3 => "Um pouquinho",
17
18             // se x for entre 4 e 10 (inclusive), exiba "um bocado":
19             4..=10 => "Um bocado",
20
21
22
23             // Tirando o que já foi definido acima, qualquer valor par
24             // exiba "Uma boa quantidade par",
25             _ if x % 2 == 0 => "Uma boa quantidade par",
26
27             // Qualquer outro valor que não se encaixou acima,
28             // exiba "Muito"
29             _ => "Muito"
30
31         });
32     }
33 }
```

19_2.rs

```rust

fn main() {

    // gerar valores para x, de 1 até 20 (inclusive):

    for x in 1..=20 {

        // Vamos exibir x e uma mensagem relacionada ao

        println!( "{}: {}", x, match x {

            // x = 1, exiba pouco:
            1 => "Pouco",

            // se x for 2 ou 3, exiba um pouquinho:
            2 | 3 => "Um pouquinho",

            // se x for entre 4 e 10 (inclusive), exiba
            4..=10 => "Um bocado",



            // Tirando o que já foi definido acima, qualquer valor par
            // exiba "Uma boa quantidade par",
            _ if x % 2 == 0 => "Uma boa quantidade par",

            // Qualquer outro valor que não se encaixou acima,
            // exiba "Muito"
            _ => "Muito"

        });
    }
}
```

```
wecomp@labwecomp:~$ rustc 19_2.rs
wecomp@labwecomp:~$ ./19_2
1: Pouco
2: Um pouquinho
3: Um pouquinho
4: Um bocado
5: Um bocado
6: Um bocado
7: Um bocado
8: Um bocado
9: Um bocado
10: Um bocado
11: Muito
12: Uma boa quantidade par
13: Muito
14: Uma boa quantidade par
15: Muito
16: Uma boa quantidade par
17: Muito
18: Uma boa quantidade par
19: Muito
20: Uma boa quantidade par
```

Antes de continuar com a linguagem propriamente dita...

Gerenciamento de memória
(bem básico...)

stack x heap

Liberação de memória

Stack (pilha):

- Área de memória organizada como uma pilha (LIFO – Last In, First Out).
- Armazena **variáveis de tamanho conhecido e de vida curta**, geralmente locais de funções.
- A alocação e liberação são **automáticas**: quando uma função termina, suas variáveis locais são removidas da pilha.
- Acesso rápido, mas limitado em tamanho.
- Bom para dados temporários e pequenas estruturas.

Heap

- Área de memória **geral para alocação dinâmica**.
- Armazena **objetos ou estruturas cujo tamanho não é conhecido em tempo de compilação** ou que precisam persistir além do escopo da função que os criou.
- A alocação e liberação podem ser **manuais ou gerenciadas por um coletor de lixo**, dependendo da linguagem.
- Mais lenta que a stack e sujeita à fragmentação, mas muito mais flexível.

Resumo da diferença principal:

- Stack → rápida, automática, limitada, para dados temporários.
- Heap → mais lenta, flexível, para dados de vida longa ou tamanho variável.

Em C...

Stack em C:

- Variáveis locais de funções são alocadas na stack.
- São automaticamente destruídas quando a função retorna.
- Exemplo:

```c
void func() {
    int x = 10;   // alocado na stack
}
```

Aqui `x` existe apenas enquanto `func` está rodando.

Heap em C:

- Para dados que precisam existir além do escopo da função ou que têm tamanho definido em tempo de execução, usamos alocação dinâmica.
- O programador aloca com `malloc` / `calloc` e libera com `free`.
- Exemplo:

```c
int* ptr = malloc(sizeof(int) * 10); // alocado na heap
// uso do ptr
free(ptr);  // liberar memória
```

Se você esquecer o `free`, ocorre **memory leak** (vaz ↓ nto de memória).

Em Java ...

- Cada thread tem sua própria stack.
- Variáveis **primitivas locais** (int, boolean, etc.) e **referências a objetos** (não os objetos em si) são armazenadas na stack.
- A stack é **automática**: quando uma função/método termina, suas variáveis locais são removidas.

Heap em Java:

- Todos os **objetos** (instâncias de classes) são alocados na heap.
- Diferente de C, você **não precisa liberar manualmente**: o **garbage collector** (coletor de lixo) identifica objetos que não têm mais referências e libera a memória automaticamente.
- Exemplo:

java Copiar código

```java
void exemplo() {
    int x = 5;               // stack
    Pessoa p = new Pessoa(); // p (referência) na stack, objeto Pessoa na heap
}
```

Quando o método termina:

- `x` e `p` desaparecem da stack.
- O objeto `Pessoa` permanece na heap até que o garbage collector perceba que não há mais referências a ele.

Resumo:

- Stack → armazenamento de variáveis locais e referências, automático.
- Heap → armazenamento de objetos, gerenciado automaticamente pelo coletor de lixo.

Em Rust...

Em Rust, os conceitos de **stack** e **heap** existem como em C e Java, mas o gerenciamento de memória é **seguro e sem garbage collector**, usando o conceito de **ownership**.

Stack em Rust:

- Variáveis de **tamanho conhecido em tempo de compilação** são armazenadas na stack.
- São liberadas automaticamente quando saem do escopo (scope).
- Exemplo:

rust Copiar código

```rust
fn main() {
    let x = 10; // alocado na stack
} // x é removido automaticamente aqui
```

Heap em Rust:

- Usada para dados cujo **tamanho não é conhecido em tempo de compilação** ou que precisam **persistir além do escopo de uma função**.
- Objetos são alocados usando `Box`, `Vec`, `String`, etc.
- Exemplo:

rust — Copiar código

```rust
fn main() {
    let s = String::from("olá"); // objeto na heap, referência na stack
} // s é removido automaticamente aqui, memória da heap liberada
```

Diferença chave em Rust:

- Não há garbage collector.
- O compilador garante que **cada pedaço de memória tem exatamente um dono (ownership)**.
- Quando o dono sai do escopo, a memória é liberada automaticamente.
- Movimentações de ownership ou referências (&, &mut) permitem controlar acesso seguro à heap sem precisar de coleta de lixo.

Resumo:

- Stack → armazenamento rápido de dados locais e de tamanho fixo, liberado ao sair do escopo.
- Heap → armazenamento de dados dinâmicos, liberado automaticamente pelo sistema de ownership do Rust.

Comparando mais diretamente:

Garbage Colector (Java)
x
Ownership (Rust)

Garbage Collector do Java vs. Ownership do Rust

Quando falamos em linguagens de programação modernas, um dos grandes desafios é **gerenciar a memória** de forma segura e eficiente. Java e Rust são dois exemplos de linguagens que resolveram esse problema de maneiras bem diferentes.

Java e o Garbage Collector

No Java, o programador **não precisa se preocupar diretamente em liberar memória**. O Java usa um componente chamado **Garbage Collector (GC)**, que roda em segundo plano e identifica objetos que não estão mais sendo utilizados. Quando o GC encontra esses objetos "órfãos", ele libera a memória que eles ocupavam.

- **Vantagens:**
  - Facilita a vida do programador, que não precisa se preocupar em desalocar memória manualmente.
  - Reduz riscos de erros como *memory leaks* (vazamentos de memória) ou *dangling pointers* (ponteiros para regiões inválidas).
- **Desvantagens:**
  - O Garbage Collector pode rodar em momentos inesperados, causando pausas na execução do programa.
  - Existe um custo extra de desempenho, já que o sistema precisa analisar constantemente quais objetos ainda estão em uso.

Rust e o Ownership

Rust resolveu o problema de outra forma: **não existe Garbage Collector**. Em vez disso, a linguagem introduz o conceito de **ownership (propriedade)**.

Cada pedaço de memória tem exatamente um **dono**, e quando esse dono sai de escopo, a memória é liberada automaticamente. Além disso, Rust define regras claras para **empréstimos (borrowing)** e **referências**, garantindo que você não use memória já liberada e evitando concorrência insegura.

- **Vantagens:**
  - O gerenciamento de memória é feito em tempo de compilação, sem precisar de um processo rodando em segundo plano.
  - Isso resulta em **programas rápidos e previsíveis**, com desempenho comparável ao de linguagens como C e C++.
  - Reduz drasticamente erros de memória, já que o compilador não deixa você compilar código inseguro.
- **Desvantagens:**
  - A curva de aprendizado é mais íngreme: o programador precisa entender e respeitar as regras de ownership e borrowing.
  - Em alguns casos, pode ser necessário reorganizar o código para se adequar às restrições impostas pelo compilador.

Comparação Final

- No **Java**, o GC facilita a programação, mas pode introduzir atrasos e perda de performance em aplicações críticas.
- No **Rust**, o ownership exige mais disciplina do programador, mas em troca oferece **alta performance e segurança de memória sem custo extra em tempo de execução**.

Em resumo: o Java aposta na **conveniência com algum custo de desempenho**, enquanto o Rust aposta na **segurança e eficiência, mesmo que o programador precise aprender um pouco mais de teoria logo no começo**.

Strings em Rust

Strings em Rust

1. Tipos básicos de dados em Rust

Rust é uma linguagem fortemente tipada e segura em relação à memória. Alguns tipos primitivos importantes:

- **Números inteiros** (`i32`, `u32`, etc.)
- **Números de ponto flutuante** (`f32`, `f64`)
- **Booleanos** (`bool`)
- **Caracteres** (`char`) — representam um único caractere Unicode (e não apenas ASCII).
  Exemplo:

rust

Copiar

```rust
let c: char = 'á';
```

Além desses tipos primitivos, temos coleções mais complexas, entre elas as **strings**.

2. Texto em Rust

Em muitas linguagens, "string" é um tipo único. Em Rust, temos **dois tipos principais para representar texto**:

1. `&str` (string slice, ou "string literal")
    - É um **trecho de string imutável**.
    - Geralmente escrita entre aspas (`"texto"`).
    - O conteúdo fica embutido no programa (normalmente na **stack** ou na área de dados estáticos).
    - Não pode crescer ou ser modificado.

2. Texto em Rust

Em muitas linguagens, "string" é um tipo único. Em Rust, temos **dois tipos principais para representar texto**:

1. `&str` (string slice, ou "string literal")
   - É um **trecho de string imutável**.
   - Geralmente escrita entre aspas (`"texto"`).
   - O conteúdo fica embutido no programa (normalmente na **stack** ou na área de dados estáticos).
   - Não pode crescer ou ser modificado.

2. `String` (tipo de dado no heap)
   - É uma **string dinâmica**, armazenada no **heap**.
   - Pode crescer e ser modificada.
   - É um objeto mais "pesado", mas muito mais flexível.

3. Diferença entre `&str` e `String`

Característica	`&str` (literal)	`String` (heap)
Mutabilidade	Sempre imutável	Pode ser mutável ou imutável
Armazenamento	Stack (ou dados estáticos)	Heap
Tamanho	Fixo	Dinâmico, pode crescer
Uso comum	Texto fixo, mensagens de erro, rótulos	Texto construído em tempo de execução, entrada de usuário

OBS:

Alguns códigos-fonte daqui pra frente terão no seu final a função **`type_of`**, para nos ajudar a ver os tipos das variáveis.

Não faz parte desta aula introdutório analisar a fundo seu funcionamento.

```
////////////////////////////////////////////////////////////////////////
//
// A função abaixo serve para ver o tipo de dados de uma variável.
// Não vamos estudar nesse momomento! Está aqui só pra ajudar a gente.

fn type_of<T>(_: &T) -> &'static str {
    std::any::type_name::<T>()
}

////////////////////////////////////////////////////////////////////////
```

20.rs

```
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = "instituto";

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: &str = "federal";

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imútavel.
    // Logo, se descomentada, a linha abaixo dá erro,
    // pois não podemos mudar o conteúdo de s2:
    // s2 = "de";


    // Podemos usar mut:

    let mut s3: &str = "sao paulo";

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = "sao carlos";

    println!("s3: {}, tipo {}", s3, type_of(&s3));

}
```

20.rs

```
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = "instituto";

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: &str = "federal";

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imútavel.
    // Logo, se descomentada, a linha abaixo dá erro,
    // pois não podemos mudar o conteúdo de s2:
    // s2 = "de";


    // Podemos usar mut:

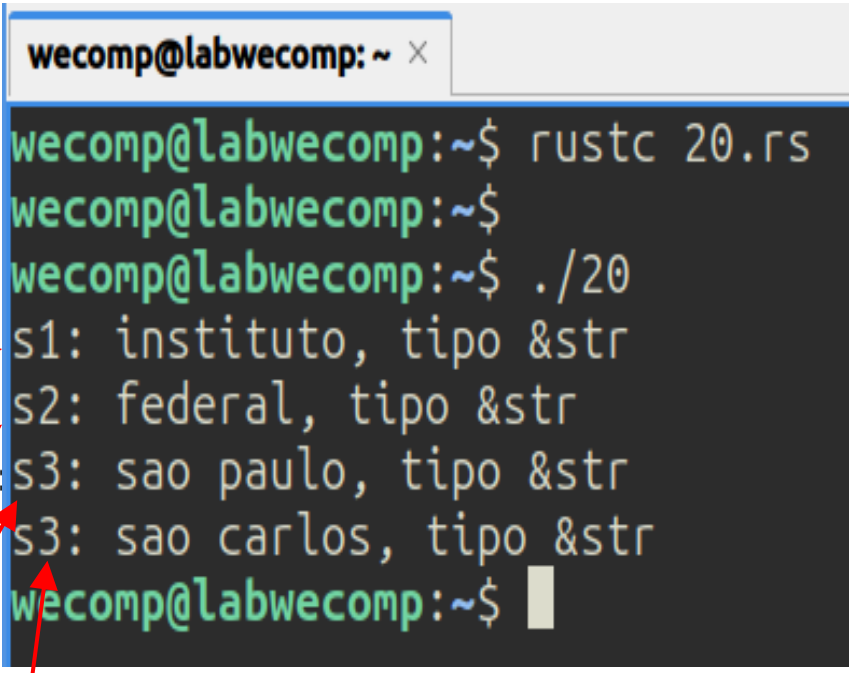
    let mut s3: &str = "sao paulo";

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = "sao carlos";

    println!("s3: {}, tipo {}", s3, type_of(&s3));

}
```



20.rs

```
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = "instituto";

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: &str = "federal";

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imútavel.
    // Logo, se descomentada, a linha abaixo dá erro,
    // pois não podemos mudar o conteúdo de s2:
       s2 = "de";


    // Podemos usar mut:

    let mut s3: &str = "sao paulo";

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = "sao carlos";

    println!("s3: {}, tipo {}", s3, type_of(&s3));

}
```



Retirando o comentário...

20.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = "instituto";

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: &str = "federal";

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imútavel.
    // Logo, se descomentada, a linha abaixo dá erro,
    // pois não podemos mudar o conteúdo de s2:
       s2 = "de";


    // Podemos usa

    let mut s3: &s

    println!("s3:

    s3 = "sao carl

    println!("s3:

}
```

```
error[E0384]: cannot assign twice to immutable variable `s2`
  --> 20.rs:17:5
   |
10 |     let s2: &str = "federal";
   |         -- first assignment to `s2`
...
17 |     s2 = "de";
   |     ^^^^^^^^^ cannot assign twice to immutable variable
   |
help: consider making this binding mutable
   |
10 |     let mut s2: &str = "federal";
   |         +++
```

O mesmo exemplo, agora com o ‘objeto’ String.

5. Criação de Strings

Formas comuns de criar `String`:

```rust
let s = String::new();                  // vazia
let s = String::from("olá");            // a partir de um literal
let s = "olá".to_string();              // converter &str -> String
```

O método `.push_str()` da `String` em Rust serve para **concatenar** um `&str` ao final da `String`.

Ele **não retorna** uma nova `String` — em vez disso, modifica a existente (se for mutável).

21.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = String::from("instituto");

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: String = String::from("federal");

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imutável.
    // Logo, se descomentadas, as linhas abaixo dão erro,
    // pois não podemos mudar o conteúdo de s2 (o 'objeto' string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usar mut:
    let mut s3: String = String::from("sao paulo");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3.push_str(" - campus sao carlos");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = String::from("VASCAO!!!");

    println!("s3: {}, tipo {}", s3, type_of(&s3));
}
```

21.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = String::from("instituto");

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: String = String::from("federal");

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imutável.
    // Logo, se descomentadas, as linhas abaixo dão erro,
    // pois não podemos mudar o conteúdo de s2 (o 'objeto' string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usa
    let mut s3: St

    println!("s3:

    s3.push_str("

    println!("s3:

    s3 = String::f

    println!("s3:
}
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 21.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./21

s1: instituto, tipo alloc::string::String
s2: federal, tipo alloc::string::String
s3: sao paulo, tipo alloc::string::String
s3: sao paulo - campus sao carlos, tipo alloc::string::String
s3: VASCAO!!!, tipo alloc::string::String
wecomp@labwecomp:~$
```

21.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = String::from("instituto");

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: String = String::from("federal");

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imutável.
    // Logo, se descomentadas, as linhas abaixo dão erro,
    // pois não podemos mudar o conteúdo de s2 (o 'objeto' string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usa
    let mut s3: St

    println!("s3:

    s3.push_str("

    println!("s3:

    s3 = String::f

    println!("s3:
}
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 21.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./21

s1: instituto, tipo alloc::string::String
s2: federal, tipo alloc::string::String
s3: sao paulo, tipo alloc::string::String
s3: sao paulo - campus sao carlos, tipo alloc::string::String
s3: VASCAO!!!, tipo alloc::string::String
wecomp@labwecomp:~$
```

21.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = String::from("instituto");

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: String = String::from("federal");

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imutável.
    // Logo, se descomentadas, as linhas abaixo dão erro,
    // pois não podemos mudar o conteúdo de s2 (o 'objeto' string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usar mut:
    let mut s3: String = String::from("sao paulo");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3.push_str(" - campus sao carlos");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = String::from("VASCAO!!!");

    println!("s3: {}, tipo {}", s3, type_of(&s3));
}
```

21.rs

```rust
fn main() {

    // Como já vimo
    let s1 = String

    println!("s1: {

    // Aqui vamos d
    let s2: String

    println!("s2: {

    // Lembrando, p
    // Logo, se des
    // pois não podemos mudar o conteúdo de s2 (o "objeto" string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usar mut:
    let mut s3: String = String::from("sao paulo");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3.push_str(" - campus sao carlos");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = String::from("VASCAO!!!");

    println!("s3: {}, tipo {}", s3, type_of(&s3));
}
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 21.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./21

s1: instituto, tipo alloc::string::String
s2: federal, tipo alloc::string::String
s3: sao paulo, tipo alloc::string::String
s3: sao paulo - campus sao carlos, tipo alloc::string::String
s3: VASCAO!!!, tipo alloc::string::String
wecomp@labwecomp:~$
```

21.rs

```rust
fn main() {

    // Como já vimo
    let s1 = String

    println!("s1: {

    // Aqui vamos d
    let s2: String

    println!("s2: {

    // Lembrando, p
    // Logo, se des
    // pois não podemos mudar o conteúdo de s2 (o "objeto" string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usar mut:
    let mut s3: String = String::from("sao paulo");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3.push_str(" - campus sao carlos");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = String::from("VASCAO!!!");

    println!("s3: {}, tipo {}", s3, type_of(&s3));
}
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 21.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./21

s1: instituto, tipo alloc::string::String
s2: federal, tipo alloc::string::String
s3: sao paulo, tipo alloc::string::String
s3: sao paulo - campus sao carlos, tipo alloc::string::String
s3: VASCAO!!!, tipo alloc::string::String
wecomp@labwecomp:~$
```

21.rs

```
wecomp@labwecomp:~$ rustc 21.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./21

s1: instituto, tipo alloc::string::String
s2: federal, tipo alloc::string::String
s3: sao paulo, tipo alloc::string::String
s3: sao paulo - campus sao carlos, tipo alloc::string::String
s3: VASCAO!!!, tipo alloc::string::String
wecomp@labwecomp:~$
```

```rust
fn main() {

    // Como já vimo
    let s1 = String

    println!("s1: {

    // Aqui vamos d
    let s2: String

    println!("s2: {

    // Lembrando, p
    // Logo, se des
    // pois não podemos mudar o conteúdo de s2 (o "objeto" string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usar mut:
    let mut s3: String = String::from("sao paulo");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3.push_str(" - campus sao carlos");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = String::from("VASCAO!!!");

    println!("s3: {}, tipo {}", s3, type_of(&s3));
}
```



21.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = String::from("instituto");

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: String = String::from("federal");

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imutável.
    // Logo, se descomentadas, as linhas abaixo dão erro,
    // pois não podemos mudar o conteúdo de s2 (o 'objeto' string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");

    // Podemos usar mut:
    let mut s3: String = String::from("sao paulo");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3.push_str(" - campus sao carlos");

    println!("s3: {}, tipo {}", s3, type_of(&s3));

    s3 = String::from("VASCAO!!!");

    println!("s3: {}, tipo {}", s3, type_of(&s3));
}
```

21.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = String::from("instituto");

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: String = String::from("federal");

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imutável.
    // Logo, se descomentadas, as linhas abaixo dão erro,
    // pois não podemos mudar o conteúdo de s2 (o 'objeto' string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");


    // Podemos usar mut:
    let mut s3: Stri
    println!("s3: {}
    s3.push_str(" -
    println!("s3: {}
    s3 = String::fro
    println!("s3: {}, tipo {}", s3, type_of(&s3));
}
```

```
wecomp@labwecomp:~$ rustc 21.rs
error[E0596]: cannot borrow `s2` as mutable, as it is not declared as mutable
  --> 21.rs:19:5
   |
19 |     s2.push_str(" SP");
   |     ^^ cannot borrow as mutable
   |
help: consider changing this to be mutable
   |
9  |     let mut s2: String = String::from("federal");
   |         +++
```

21.rs

```rust
fn main() {

    // Como já vimos, o Rust infere qual o tipo:
    let s1 = String::from("instituto");

    println!("s1: {}, tipo {}", s1, type_of(&s1));

    // Aqui vamos dizer explicitamente qual o tipo:
    let s2: String = String::from("federal");

    println!("s2: {}, tipo {}", s2, type_of(&s2));

    // Lembrando, por default, a variável é imutável.
    // Logo, se descomentadas, as linhas abaixo dão erro,
    // pois não podemos mudar o conteúdo de s2 (o 'objeto' string),
    // nem para onde s2 está apontando (uma outra string, p.ex.):
    //
    // Trocando o conteúdo da string apontada por s2:
    // s2.push_str(" SP");
    //
    // Trocando totalmente o objeto apontado por s2 (para um novo objeto):
    // s2 = String::from("SP");

    // Podemos usar mut:
    let mut s3: String = String::fr

    println!("s3: {}, tipo {}", s3,

    s3.push_str(" - campus sao carl

    println!("s3: {}, tipo {}", s3,

    s3 = String::from("VASCAO!!!");

    println!("s3: {}, tipo {}", s3,
}
```

```
error[E0384]: cannot assign twice to immutable variable `s2`
  --> 21.rs:22:5
   |
9  |     let s2: String = String::from("federal");
   |         -- first assignment to `s2`
...
22 |     s2 = String::from("SP");
   |     ^^ cannot assign twice to immutable variable
   |
help: consider making this binding mutable
   |
9  |     let mut s2: String = String::from("federal");
   |         +++
```

Entrando na parte divertida...



22.rs

```
1
2  // Função para exibir uma string criada na stack (&str)
3  // recebida por parâmetro:
4
5  fn exibe_string( s: &str ) {
6
7      println!( "{}", s );
8
9  }
10
11
12 fn main() {
13
14     // String alocada na stack (&str):
15
16     let uma_string = "IFSP";
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21 }
22
```

22.rs

```
1
2 // Função para exibir uma string criada na stack (&str)
3 // recebida por parâmetro:
4
5 fn exibe_string( s: &str ) {
6
7     println!( "{}", s );
8
9 }
10
11
12 fn main() {
13
14     // String alocada na stack (&str):
15
16     let uma_string = "IFSP";
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21 }
22
```

```
wecomp@labwecomp:~$ rustc 22.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./22
IFSP
wecomp@labwecomp:~$
```

Vamos exibir a string também no main ?

23.rs

```
 1
 2 // Função para exibir uma string criada na stack (&str)
 3 // recebida por parâmetro:
 4
 5 fn exibe_string( s: &str ) {
 6
 7     println!( "{}", s );
 8
 9 }
10
11
12 fn main() {
13
14     // String alocada na stack (&str):
15
16     let uma_string = "IFSP";
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21     // Exibindo direto por aqui também:
22     println!( "{}", uma_string );
23
24 }
```

23.rs

```
1
2  // Função para exibir uma string criada na stack (&str)
3  // recebida por parâmetro:
4
5  fn exibe_string( s: &str ) {
6
7      println!( "{}", s );
8
9  }
10
11
12 fn main() {
13
14     // String alocada na stack (&str):
15
16     let uma_string = "IFSP";
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21     // Exibindo direto por aqui também:
22     println!( "{}", uma_string );
23
24 }
```

```
wecomp@labwecomp:~$ rustc 23.rs
wecomp@labwecomp:~$ ./23
IFSP
IFSP
wecomp@labwecomp:~$
```

23.rs

```
1
2  // Função para exibir uma string criada na stack (&str)
3  // recebida por parâmetro:
4
5  fn exibe_string( s: &str ) {
6
7      println!( "{}", s );
8
9  }
10
11
12 fn main() {
13
14     // String alocada na stack (&str):
15
16     let uma_string = "IFSP";
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21     // Exibindo direto por aqui também:
22     println!( "{}", uma_string );
23
24 }
```

```
wecomp@labwecomp:~$ rustc 23.rs
wecomp@labwecomp:~$ ./23
IFSP
IFSP
wecomp@labwecomp:~$
```



Calma, calma.... 

Vamos mudar para o tipo String
(como já vimos, armazenado na heap).

24.rs

```
1
2  // Função para exibir uma string criada na heap (String)
3  // recebida por parâmetro:
4
5  fn exibe_string( s: String ) {
6
7      println!( "{}", s );
8
9  }
10
11
12 fn main() {
13
14     // String alocada na heap (String):
15
16     let uma_string = String::from("IFSP");
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21 }
22
```

24.rs

```
1
2 // Função para exibir uma string criada na heap (String)
3 // recebida por parâmetro:
4
5 fn exibe_string( s: String ) {
6
7     println!( "{}", s );
8
9 }
10
11
12 fn main() {
13
14     // String alocada na heap (String):
15
16     let uma_string = String::from("IFSP");
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21 }
22
```

```
wecomp@labwecomp:~$ rustc 24.rs
wecomp@labwecomp:~$ ./24
IFSP
wecomp@labwecomp:~$
```

24.rs

```
1
2 // Função para exibir uma string criada na heap (String)
3 // recebida por parâmetro:
4
5 fn exibe_string( s: String ) {
6
7     println!( "{}", s );
8
9 }
10
11
12 fn main() {
13
14     // String alocada na heap (String):
15
16     let uma_string = String::from("IFSP");
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21 }
22
```

```
wecomp@labwecomp:~$ rustc 24.rs
wecomp@labwecomp:~$ ./24
IFSP
wecomp@labwecomp:~$
```



Vamos, novamente,
exibir a string também no main:

25.rs

```
1
2  // Função para exibir uma string criada na heap (String)
3  // recebida por parâmetro:
4
5  fn exibe_string( s: String ) {
6
7      println!( "{}", s );
8
9  }
10
11
12 fn main() {
13
14     // String alocada na heap (String):
15
16     let uma_string = String::from("IFSP");
17
18     // Chamando a função acima pra exibir:
19     exibe_string( uma_string );
20
21     // Exibindo direto por aqui também:
22     println!( "{}", uma_string );
23
24 }
```

25.rs

```
1
2 // Função para exibir uma string criada na heap (String)
3 // recebida por parâmetro:
4
5 fn exibe_string( s: String ) {
6
7     println!( "{}", s );
8
9 }
10
11
12 fn ma
13
14     /
15
16     l
17
18     /
19     e
20
21     /
22     p
23
24 }
```

```
wecomp@labwecomp:~$ rustc 25.rs
error[E0382]: borrow of moved value: `uma_string`
  --> 25.rs:22:21
   |
16 |     let uma_string = String::from("IFSP");
   |         ---------- move occurs because `uma_string` has type `String`, which
 does not implement the `Copy` trait
...
19 |     exibe_string( uma_string );
   |                   ---------- value moved here
...
22 |     println!( "{}", uma_string );
   |                     ^^^^^^^^^^ value borrowed here after move
   |
note: consider changing this parameter type in function `exibe_string` to borrow
instead if owning the value isn't necessary
  --> 25.rs:5:21
   |
5  | fn exibe_string( s: String ) {
   |    ------------     ^^^^^^ this parameter takes ownership of the value
   |    |
   |    in this function
```

25.rs

```
1
2 // Função para exibir uma string criada na heap (String)
3 // recebida por parâmetro:
4
5 fn exibe_string( s: String ) {
6
7     println!( "{}", s );
8
9 }
10
11
12 fn ma
13
14     /
15
16     l
17
18     /
19     e
20
21     /
22     p
23
24 }
```

```
wecomp@labwecomp:~$ rustc 25.rs
error[E0382]: borrow of moved value: `uma_string`
  --> 25.rs:22:21
   |
16 |     let uma_string = String::from("IFSP");
   |         ---------- move occurs because `uma_string` has type `String`, which
 does not implement the `Copy` trait
...
19 |     exibe_string( uma_string );
   |                   ---------- value moved here
...
22 |     println!( "{}", uma_string );
   |                     ^^^^^^^^^^ value borrowed her
   |
note: consider changing this parameter type in functio
instead if owning the value isn't necessary
  --> 25.rs:5:21
   |
5  | fn exibe_string( s: String ) {
   |    ------------     ^^^^^^ this parameter takes o
   |    |
   |    in this function
```



25.rs

```
1
2 // Função para exibir uma string criada na heap (String)
3 // recebida por parâmetro:
4
5 fn exibe_string( s: String ) {
6
7     println!( "{}", s );
8
9 }
10
11
12 fn ma
13
14     /
15
16     l
17
18     /
19     e
20
21     /
22     p
23
24 }
```

```
wecomp@labwecomp:~$ rustc 25.rs
error[E0382]: borrow of moved value: `uma_string`
  --> 25.rs:22:21
   |
16 |     let uma_string = String::from("IFSP");
   |         ---------- move occurs because `uma_
   |  does not implement the `Copy` trait
...
19 |     exibe_string( uma_string );
   |                   ---------- value moved her
...
22 |     println!( "{}", uma_string );
   |                     ^^^^^^^^^^ value borrowed here after move
   |
note: consider changing this parameter type in function `exibe_string` to borrow
instead if owning the value isn't necessary
  --> 25.rs:5:21
   |
5  | fn exibe_string( s: String ) {
   |    ------------     ^^^^^^ this parameter takes ownership of the value
   |    |
   |    in this function
```



25.rs

```rust
// Função para exibir uma string criada na heap (String)
// recebida por parâmetro:

fn exibe_string( s: String ) {

    println!( "{}", s );

}


fn main() {

    // String alocada na heap (String):

    let uma_string = String::from("IFSP");

    // Chamando a função acima pra exibir:
    exibe_string( uma_string );

    // Exibindo direto por aqui também:
    println!( "{}", uma_string );

}
```

Nesse programa, o erro acontece porque, quando chamamos a função `exibe_string(uma_string)`, o **Rust entende que a variável `uma_string` deixou de pertencer ao `main` e passou a pertencer à função.**

Ou seja: em Rust, cada valor tem sempre **um único dono** (*owner*). No momento em que passamos `uma_string` para a função, a função passa a ser a dona dela. Como a função não devolve a string, quando ela termina, a string é descartada.

Depois disso, no `main`, quando tentamos usar `uma_string` de novo, o compilador reclama, porque aquela variável **já não é mais dona do valor** – o valor foi "movido" para a função.

Essa regra de "cada valor tem um dono" é o que chamamos de **ownership** no Rust. É a base da forma como a linguagem garante segurança de memória sem precisar de coletor de lixo (*garbage collector*).

25.rs

```rust

// Função para exibir uma string criada na heap (String)
// recebida por parâmetro:

fn exibe_string( s: String ) {

    println!( "{}", s );

}


fn main() {

    // String alocada na heap (String):

    let uma_string = String::from("IFSP");

    // Chamando a função acima pra exibir:
    exibe_string( uma_string );

    // Exibindo direto por aqui também:
    println!( "{}", uma_string );

}
```

Nesse programa, o erro acontece porque, quando chamamos a função `exibe_string(uma_string)`, o **Rust entende que a variável `uma_string` deixou de pertencer ao `main` e passou a pertencer à função.**

Ou seja: em Rust, cada valor tem sempre **um único dono** (*owner*). No momento em que passamos `uma_string` para a função, a função passa a ser a dona dela. Como a função não devolve a string, quando ela termina, a string é descartada.

Depois disso, no `main`, quando tentamos usar `uma_string` de novo, o compilador reclama, porque aquela variável **já não é mais dona do valor** – o valor foi "movido" para a função.

Essa regra de "cada valor tem um dono" é o que chamamos de **ownership** no Rust. É a base da forma como a linguagem garante segurança de memória sem precisar de coletor de lixo (*garbage collector*).

25.rs

```rust

// Função para exibir uma string criada na heap (String)
// recebida por parâmetro:

fn exibe_string( s: String ) {

    println!( "{}", s );

}


fn main() {

    // String alocada na heap (String):

    let uma_string = String::from("IFSP");

    // Chamando a função acima pra exibir:
    exibe_string( uma_string );

    // Exibindo direto por aqui também:
    println!( "{}", uma_string );

}
```

Nesse programa, o erro acontece porque, quando chamamos a função `exibe_string(uma_string)`, o **Rust entende que a variável `uma_string` deixou de pertencer ao `main` e passou a pertencer à função.**

Ou seja: em Rust, cada valor tem sempre **um único dono** (*owner*). No momento em que passamos `uma_string` para a função, a função passa a ser a dona dela. Como a função não devolve a string, quando ela termina, a string é descartada.

Depois disso, no `main`, quando tentamos usar `uma_string` de novo, o compilador reclama, porque aquela variável **já não é mais dona do valor** – o valor foi "movido" para a função.

Essa regra de "cada valor tem um dono" é o que chamamos de **ownership** no Rust. É a base da forma como a linguagem garante segurança de memória sem precisar de coletor de lixo (*garbage collector*).

Como poderíamos resolver ?

1) Devolvendo a String para o main...

26.rs - Devolvendo a string para o main...

```
1
2  // Função para exibir uma string criada na heap (String)
3  // recebida por parâmetro:
4
5  fn exibe_string( s: String ) -> String {
6
7      println!( "{}", s );
8
9      // Lembrando, sem ; é uma expressão, retorna um valor.
10     // Vamos retornar a String s:
11
12     s
13
14 }
15
16
17 fn main() {
18
19     // String alocada na heap (String):
20
21     let uma_string = String::from("IFSP");
22
23     // Chamando a função e recuperando a String no final:
24     let uma_string = exibe_string( uma_string );
25
26     // Exibindo direto por aqui também:
27     println!( "{}", uma_string );
28
29 }
30
```

26.rs - Devolvendo a string para o main...

```
1
2  // Função para exibir uma string criada na heap (String)
3  // recebida por parâmetro:
4
5  fn exibe_string( s: String ) -> String {
6
7      println!( "{}", s );
8
9      // Lembrando, sem ; é uma expressão, retorna um valor.
10     // Vamos retornar a String s:
11
12     s
13
14 }
15
16
17 fn main() {
18
19     // String alocada na heap (String):
20
21     let uma_string = String::from("IFSP");
22
23     // Chamando a função e recuperando a String no final:
24     let uma_string = exibe_string( uma_string );
25
26     // Exibindo direto por aqui também:
27     println!( "{}", uma_string );
28
29 }
30
```

```
wecomp@labwecomp:~$ rustc 26.rs
wecomp@labwecomp:~$ ./26
IFSP
IFSP
wecomp@labwecomp:~$
```

26.rs - Devolvendo a string para o main...

```
 1
 2 // Função para exibir uma string criada na heap (String)
 3 // recebida por parâmetro:
 4
 5 fn exibe_string( s: String ) -> String {
 6
 7     println!( "{}", s );
 8
 9     // Lembrando, sem ; é uma expressão, retorna um valor.
10     // Vamos retornar a String s:
11
12     s
13
14 }
15
16
17 fn main() {
18
19     // String alocada na heap (String):
20
21     let uma_string = String::from("IFSP");
22
23     // Chamando a função e recuperando a String no
24     let uma_string = exibe_string( uma_string );
25
26     // Exibindo direto por aqui também:
27     println!( "{}", uma_string );
28
29 }
30
```

Mas péra....
Se não é `mut`, por que funcionou ???



26.rs - Devolvendo a string para o main...

```
1
2  // Função par
3  // recebida p
4
5  fn exibe_stri
6
7      println!(
8
9      // Lembra
10     // Vamos
11
12     s
13
14 }
15
16
17 fn main() {
18
19     // String alocada na heap (String):
20
21     let uma_string = String::from("IFSP");
22
23     // Chamando a função e recuperando a String no final:
24     let uma_string = exibe_string( uma_string );
25
26     // Exibindo direto por aqui também:
27     println!( "{}", uma_string );
28
29 }
30
```

Isso acontece por causa de **shadowing** em Rust.

No Rust, quando você escreve:

rust — Copiar código

```rust
let uma_string = exibe_string(uma_string);
```

não estamos modificando a variável original; estamos **criando uma nova variável com o mesmo nome**, que "sombra" (`shadow`) a anterior. A primeira `uma_string` deixou de existir, e a nova passa a ser usada no restante do código.

Por isso **não precisamos do** `mut`: não estamos alterando a variável existente, apenas criando outra com o mesmo nome.

2) Passando uma referência para a função, assim o main continua sendo dono da string e não ocorre o erro.

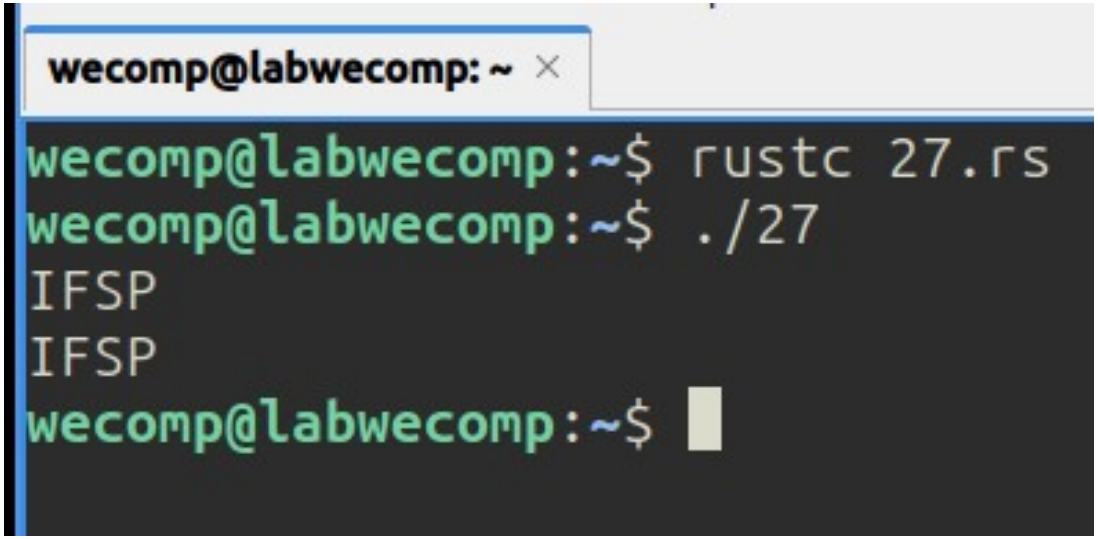
Em Rust, referências são indicadas com &

27.rs - Passando uma referência...

```
1
2  // Função para exibir uma string
3  // recebendo sua referência:
4
5  fn exibe_string( s: &String ) {
6
7      println!( "{}", s );
8
9  }
10
11
12 fn main() {
13
14     // String alocada na heap (String):
15
16     let uma_string = String::from("IFSP");
17
18     // Passando referência, mas o main ainda é dono:
19     exibe_string( &uma_string );
20
21     // Exibindo direto por aqui também:
22     println!( "{}", uma_string );
23
24 }
```

27.rs - Passando uma referência...

```
1
2  // Função para exibir uma string
3  // recebendo sua referência:
4
5  fn exibe_string( s: &String ) {
6
7      println!( "{}", s );
8
9  }
10
11
12 fn main() {
13
14     // String alocada na heap (String):
15
16     let uma_string = String::from("IFSP");
17
18     // Passando referência, mas o main ainda é dono:
19     exibe_string( &uma_string );
20
21     // Exibindo direto por aqui também:
22     println!( "{}", uma_string );
23
24 }
```



Mas...

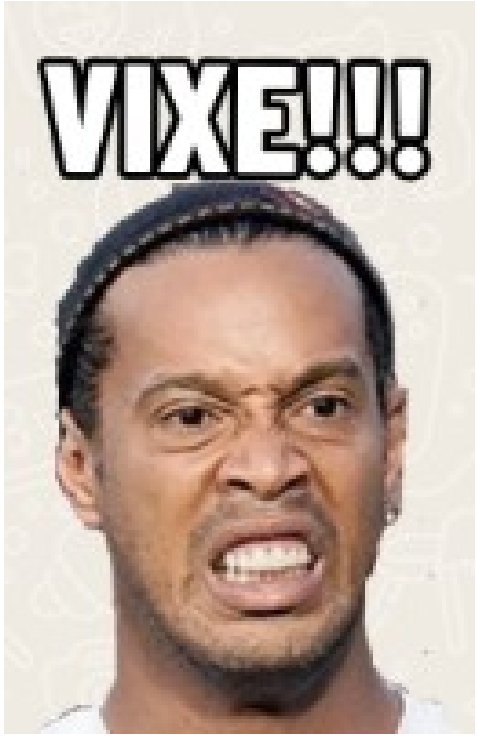


28.rs

```
1
2  // Função para exibir uma string
3  // recebendo sua referência:
4
5  fn exibe_string( s: &String ) {
6
7      println!( "{}", s );
8
9      // Concatenando algo no final:
10     s.push_str(" - São Carlos");
11
12 }
13
14
15 fn main() {
16
17     // String alocada na heap (String):
18
19     let uma_string = String::from("IFSP");
20
21     // Passando referência, mas o main ainda é dono:
22     exibe_string( &uma_string );
23
24     // Exibindo direto por aqui também:
25     println!( "{}", uma_string );
26
27 }
```

28.rs

```
1
2  // Função para exibir uma string
3  // recebendo sua referência:
4
5  fn exibe_string( s: &String ) {
6
7      println!( "{}", s );
8
9      // Concatenando algo no final:
10     s.push_str(" - São Carlos");
11
12 }
13
14
15 fn main() {
16
17
18
19
20
21
22
23
24
25
26
27 }
28
```




```
wecomp@labwecomp:~$ rustc 28.rs
error[E0596]: cannot borrow `*s` as mutable, as it is behind a `&` reference
  --> 28.rs:10:5
   |
10 |     s.push_str(" - São Carlos");
   |     ^ `s` is a `&` reference, so the data it refers to cannot be borrowed as
mutable
   |
help: consider changing this to be a mutable reference
   |
5  | fn exibe_string( s: &mut String ) {
   |                     +++
```

28.rs

```rust

// Função para exibir uma string
// recebendo sua referência:

fn exibe_string( s: &String ) {

    println!( "{}", s );

    // Concatenando algo no final:
    s.push_str(" - São Carlos");

}


fn main() {

    // String alocad

    let uma_string =

    // Passando refe
    exibe_string( &u

    // Exibindo dire
    println!( "{}",

}
```

O erro acontece porque, mesmo que a função receba uma **referência**, ela está tentando **modificar** a string com `push_str`.

Em Rust, referências **imutáveis** (`&String`) permitem **apenas ler** o valor, não alterá-lo. Então esta linha:

rust — Copiar código

```rust
s.push_str(" - São Carlos");
```

não é permitida, e o compilador reclama algo como:

javascript — Copiar código

```
error: cannot borrow `*s` as mutable, as it is behind a `&` reference
```

💡 **Resumo simples:**

- `&String` → só leitura, não pode modificar.
- Para modificar, a função precisaria receber uma **referência mutável** (`&mut String`) e a variável no `main` precisaria ser `mut`.

29.rs - Pra resolver...

```
1
2  // Função para exibir uma string
3  // recebendo sua referência MUTÁVEL:
4
5  fn exibe_string( s: &mut String ) {
6
7      println!( "{}", s );
8
9      // Concatenando algo no final:
10     s.push_str(" - São Carlos");
11
12 }
13
14
15 fn main() {
16
17     // String alocada na heap (String):
18
19     let mut uma_string = String::from("IFSP");
20
21     // Passando referência, mas o main ainda é dono:
22     exibe_string( &mut uma_string );
23
24     // Exibindo direto por aqui também:
25     println!( "{}", uma_string );
26
27 }
28
```

29.rs - Pra resolver...

```
1
2  // Função para exibir uma string
3  // recebendo sua referência MUTÁVEL:
4
5  fn exibe_string( s: &mut String ) {
6
7      println!( "{}", s );
8
9      // Concatenando algo no final:
10     s.push_str(" - São Carlos");
11
12 }
13
14
15 fn main() {
16
17     // String alocada na heap (String):
18
19     let mut uma_string = String::from("IFSP");
20
21     // Passando referência, mas o main ainda é dono:
22     exibe_string( &mut uma_string );
23
24     // Exibindo direto por aqui também:
25     println!( "{}", uma_string );
26
27 }
28
```

```
wecomp@labwecomp:~$ rustc 29.rs
wecomp@labwecomp:~$ ./29
IFSP
IFSP - São Carlos
wecomp@labwecomp:~$
```

Em Rust, **borrowing múltiplo** significa que você pode ter **várias referências imutáveis** (`&T`) para o mesmo valor ao mesmo tempo, mas **não pode misturar com referência mutável** (`&mut T`). Isso permite ler um valor em vários lugares sem transferir ownership.

💡 **Regra de ouro:**

- **Múltiplas referências imutáveis** → permitido.
- **Uma referência mutável** → permitido.
- **Não pode misturar referência mutável com referências imutáveis** ao mesmo tempo.

Variável	Referências Imutáveis (&T)	Referência Mutável (&mut T)
`let x = ...` (imutável)	✅ Permitido: várias referências ao mesmo tempo	❌ Não permitido
`let mut x = ...` (mutável)	✅ Permitido: várias referências ao mesmo tempo	✅ Permitido: apenas 1 de cada vez ❌ Sem referências imutáveis simultâneas

30.rs

```rust
// Função que recebe referência imutável e imprime

fn exibe_ref(s: &String) {
    println!("{}", s);
}

fn main() {

    // Variável imutável
    let uma_string = String::from("IFSP");

    // Variável mutável
    let mut outra_string = String::from("Campus São Carlos");
    outra_string.push_str(" - SP");

    // Duas referências imutáveis para a variável imutável
    let ref1 = &uma_string;
    let ref2 = &uma_string;

    // Duas referências imutáveis para a variável mutável
    let ref3 = &outra_string;
    let ref4 = &outra_string;

    // Chamando a função para cada referência
    exibe_ref(ref1);
    exibe_ref(ref2);
    exibe_ref(ref3);
    exibe_ref(ref4);

    println!("Podemos usar as variáveis originais normalmente:");
    println!("uma_string: {}", uma_string);
    println!("outra_string: {}", outra_string);

}
```

30.rs

```rust
// Função que recebe referência i

fn exibe_ref(s: &String) {
    println!("{}", s);
}

fn main() {

    // Variável imutável
    let uma_string = String::fro

    // Variável mutável
    let mut outra_string = String::from("Campus São Carlos");
    outra_string.push_str(" - SP");

    // Duas referências imutáveis para a variável imutável
    let ref1 = &uma_string;
    let ref2 = &uma_string;

    // Duas referências imutáveis para a variável mutável
    let ref3 = &outra_string;
    let ref4 = &outra_string;

    // Chamando a função para cada referência
    exibe_ref(ref1);
    exibe_ref(ref2);
    exibe_ref(ref3);
    exibe_ref(ref4);

    println!("Podemos usar as variáveis originais normalmente:");
    println!("uma_string: {}", uma_string);
    println!("outra_string: {}", outra_string);

}
```

```
wecomp@labwecomp:~$ rustc 30.rs
wecomp@labwecomp:~$ ./30
IFSP
IFSP
Campus São Carlos - SP
Campus São Carlos - SP
Podemos usar as variáveis originais normalmente:
uma_string: IFSP
outra_string: Campus São Carlos - SP
wecomp@labwecomp:~$
```

Misturar tipos dá erro!

31.rs

```rust

fn main() {

    let mut uma_string = String::from("IFSP");

    let ref1 = &uma_string;    // referência imutável

    let ref_mut = &mut uma_string; // ❌ erro: não pode misturar com imutável

    println!("{}", ref1);

}

```

31.rs

```rust

fn main() {

    let mut uma_string = String::from("IFSP");

    let ref1 = &uma_string;    // referência imutável

    let ref_mut = &mut uma_string; // ❌ erro: não pode misturar com imutável

    println!("{}", ref1);

}

```

```
error[E0502]: cannot borrow `uma_string` as mutable because it is also borrowed as
 immutable
  --> 31.rs:8:19
   |
6  |     let ref1 = &uma_string;    // referência imutável
   |                ----------- immutable borrow occurs here
7  |
8  |     let ref_mut = &mut uma_string; // ×  erro: não pode misturar com imutável
   |                   ^^^^^^^^^^^^^^^ mutable borrow occurs here
9  |
10 |     println!("{}", ref1);
   |                    ---- immutable borrow later used here
```

Tratamento de erros:

Result

1. O que é o `Result`

No Rust, `Result` é definido assim

rust Copiar código

```rust
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

- `T` → é o tipo do valor retornado em caso de sucesso.
- `E` → é o tipo do erro em caso de falha.

Ou seja, quando você usa `Result`, está dizendo:

"Esta função pode devolver um valor válido (`Ok`) ou um erro (`Err`)."

2. Por que usar `Result`

Diferente de linguagens com **exceções**, em Rust **os erros fazem parte do tipo de retorno**. Isso força você a lidar com eles, o que aumenta a segurança.

Exemplo real: ler um arquivo pode funcionar ou falhar (arquivo inexistente, permissão negada etc.). A biblioteca padrão usa `Result` para isso.

32.rs

```rust
// Função que divide os parâmetros, a / b.
// Se OK, devolve um f64 (o resultado).
// Se erro, devolve uma String.

fn divide(a: f64, b: f64) -> Result<f64, String> {

    if b == 0.0 {
        Err(String::from("Não é possível dividir por zero"))
    } else {
        Ok(a / b)
    }
}


fn main() {

    let resultado1 = divide(10.0, 2.0);
    let resultado2 = divide(5.0, 0.0);

    match resultado1 {
        Ok(valor) => println!("Resultado da divisão 10/2 : {}", valor),
        Err(erro) => println!("Erro: {}", erro),
    }

    match resultado2 {
        Ok(valor) => println!("Resultado da divisão 5/0 : {}", valor),
        Err(erro) => println!("Erro: {}", erro),
    }
}
```

32.rs

```rust
// Função que divide os parâmetros, a / b.
// Se OK, devolve um f64 (o resultado).
// Se erro, devolve uma String.

fn divide(a: f64, b: f64) -> Result<f64, String> {

    if b == 0.0 {
        Err(String::from("Não é possível dividir por zero"))
    } else {
        Ok(a / b)
    }
}


fn main() {

    let resultad
    let resultad

    match result
        Ok(valor
        Err(erro
    }

    match result
        Ok(valor
        Err(erro
    }
}
```

1. Cabeçalho da função

rust — Copiar código

```rust
fn divide(a: f64, b: f64) -> Result<f64, String>
```

- `divide` → nome da função.
- `(a: f64, b: f64)` → recebe dois números de ponto flutuante (`f64`).
- `-> Result<f64, String>` → retorna um `Result` :
  - `Ok(f64)` se a divisão for bem-sucedida.
  - `Err(String)` se houver erro (neste caso, uma mensagem de texto explicando o problema).

32.rs

```rust
// Função que divide os parâmetros, a / b.
// Se OK, devolve um f64 (o resultado).
// Se erro, devolve uma String.

fn divide(a: f64, b: f64) -> Result<f64, String> {

    if b == 0.0 {
        Err(String::from("Não é possível dividir por zero"))
    } else {
        Ok(a / b)
    }
}


fn main() {

    let resultado1
    let resultado2

    match resultad
        Ok(valor)
        Err(erro)
    }

    match resultad
        Ok(valor)
        Err(erro)
    }
}
```

2. Corpo da função

rust — Copiar código

```rust
if b == 0.0 {
    Err(String::from("Não é possível dividir por zero"))
} else {
    Ok(a / b)
}
```

- Se o divisor `b` for **zero**, a função retorna um `Err` com a mensagem `"Não é possível dividir por zero"`.
- Caso contrário, retorna um `Ok` contendo o valor da divisão `a / b`.

32.rs

```rust
// Função que divide os parâmetros, a / b.
// Se OK, devolve um f64 (o resultado).
// Se erro, devolve uma String.

fn divide(a: f64, b: f64) -> Result<f64, String> {

    if b == 0.0 {
        Err(String::from("Não é possível dividir por zero"))
    } else {
        Ok(a / b)
    }
}


fn main() {

    let resultado1 = divide(10.0, 2.0);
    let resultado2 = divide(5.0, 0.0);

    match resultado1 {
        Ok(valor) => println!("Resultado da divisão 10/2 : {}", valor),
        Err(erro) => println!("Erro: {}", erro),
    }

    match resultado2 {
        Ok(valor) => println!("Resultado da divisão 5/0 : {}", valor),
        Err(erro) => println!("Erro: {}", erro),
    }
}
```

```
wecomp@labwecomp:~$ rustc 32.rs
wecomp@labwecomp:~$ ./32
Resultado da divisão 10/2 : 5
Erro: Não é possível dividir por zero
wecomp@labwecomp:~$
```

“Ok, bonitinho, mas....
neste código:

```
fn divide(a: f64, b: f64) -> Result<f64, String> {

    if b == 0.0 {
        Err(String::from("Não é possível dividir por zero"))
    } else {
        Ok(a / b)
    }
}
```

você teve que tratar e filtrar **especificamente**
o aspecto do código que poderia gerar o erro (`if b == 0.0`).

Se eu estivesse em Java, eu poderia apenas capturar a exceção
“`java.lang.ArithmeticException: / by zero`”,
que seria emitida automaticamente na divisão por zero,
ao invés de olhar especificamente se o denominador é zero.

Ou seja, em Rust eu preciso identificar o erro específico
que pode dar, e responder adequadamente,
enquanto que em Java, apenas respondo a uma categoria de erros!

Resumindo:

Em Rust eu tenho que ver, na mão,
se o denominador é zero.

Em Java eu simplesmente não me importo com isso, deixo a divisão acontecer normalmente, e apenas programo a captura da exceção, para o caso dela acontecer.

Enfim....
qual a vantagem ??

Você está percebendo bem a **diferença de filosofia** entre Java e Rust. Vamos organizar:

---

1. Java — Erros como exceções automáticas

- No Java, certas situações de erro **disparam exceções automaticamente** (ex.: `10 / 0` → `ArithmeticException`).
- Você não precisa prever o erro antes. Basta colocar o código dentro de um `try/catch` e capturar.
- Isso dá praticidade, mas tem um custo: qualquer linha pode lançar uma exceção, e se não for capturada, o programa cai.

👉 Em resumo: no Java, o erro é **detectado pelo runtime** e tratado **dinamicamente**.

---

2. Rust — Erros como parte da assinatura

- Em Rust, **não existem exceções de runtime automáticas** (salvo *panics*, que não são para fluxo normal de programa).
- Se você quer lidar com um erro, precisa **modelá-lo** no retorno da função (`Result`, `Option`, etc.).
- Isso obriga a pensar: *"O que pode dar errado aqui?"* → e representar explicitamente.
- O compilador garante que quem chama a função saiba que há possibilidade de erro e trate disso.

👉 Em resumo: em Rust, o erro é **explicado pelo tipo** e tratado **estaticamente**.

3. Diferença-chave (o ponto que você levantou)

- **Java** → você não precisa prever qual erro, porque o runtime lança exceções para você.
- **Rust** → você precisa **prever e explicitar** o erro possível (divisão por zero, arquivo não encontrado, etc.).

Isso dá mais trabalho, mas aumenta a segurança: o compilador garante que nenhum erro será "ignorado sem querer".

---

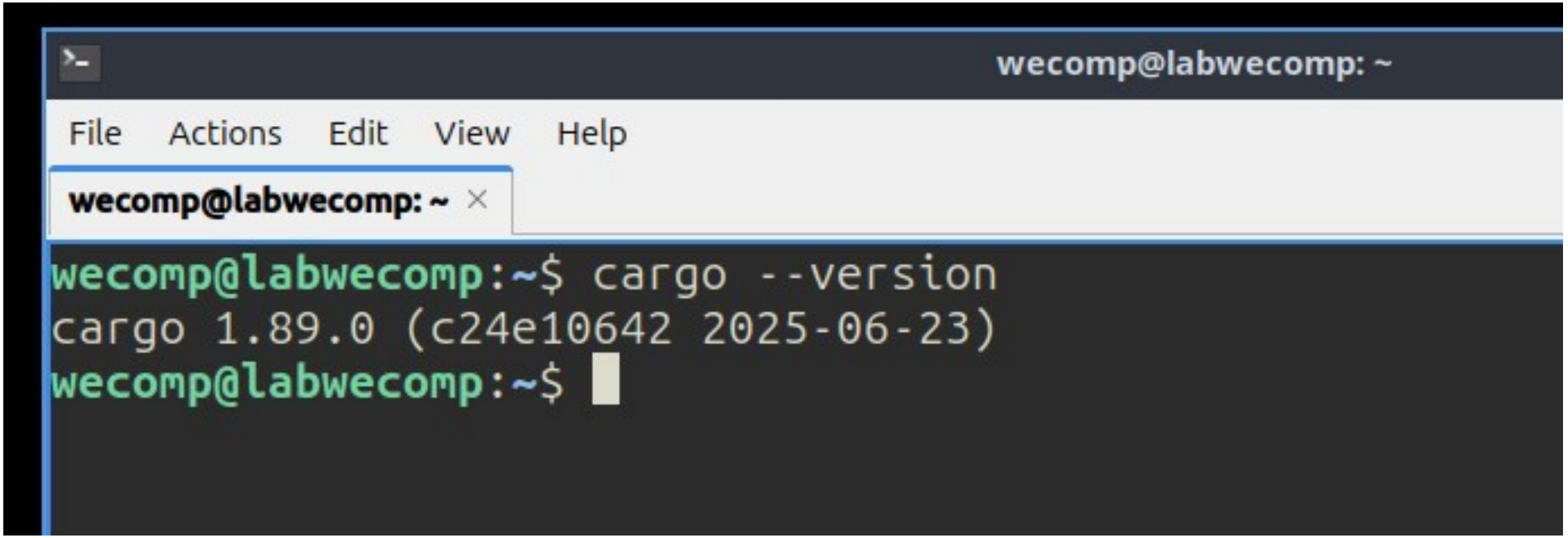
4. Consequência prática

- Em **Java**, se você esquece o `try/catch`, o programa quebra em tempo de execução.
- Em **Rust**, se você esquece de tratar um `Result`, o compilador não deixa compilar (a não ser que use `.unwrap()` ou ignore explicitamente).

📌 Então a sua leitura está correta:

- **Java** → **reativo**: não precisa filtrar, só capturar exceções.
- **Rust** → **proativo**: precisa prever os erros e representá-los.

Gerenciando projetos com cargo


wecomp@labwecomp: ~
File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$ cargo --version
cargo 1.89.0 (c24e10642 2025-06-23)
wecomp@labwecomp:~$

Cargo é o **gerenciador de pacotes e de projetos do Rust**. Ele vem junto com a instalação padrão do Rust (via *rustup*) e tem dois papéis principais:

1. **Gerenciar dependências**
   - Cargo baixa, compila e organiza as bibliotecas externas (chamadas de *crates*) que você quiser usar no projeto.
   - Ele busca as crates do repositório crates.io ↗.
2. **Gerenciar projetos Rust**
   - Permite criar, compilar, rodar e testar projetos de forma padronizada.
   - Usa dois arquivos centrais:
     - `Cargo.toml` : onde você define as dependências e metadados do projeto.
     - `Cargo.lock` : onde ficam registradas as versões exatas das dependências utilizadas.

Em Rust, uma **crate** é a **unidade de compilação e distribuição** de código.

- Pode ser uma **biblioteca** (usada como dependência em outros projetos).
- Pode ser um **executável** (um programa completo, como o que você cria com `cargo new`).

Tipos de crates

1. **Crate binária**: gera um programa executável.
   - Exemplo: quando você cria um projeto com `cargo new meu_app`, o `main.rs` é o ponto de entrada.
2. **Crate de biblioteca**: não tem `main.rs`, mas sim `lib.rs`.
   - Fornece funções, structs e traits para outros projetos usarem.
   - Exemplo: a crate `rand`, que você importou.

Estrutura

Um projeto em Rust (gerenciado pelo Cargo) sempre é uma crate.

- Arquivo principal: `src/main.rs` (se for binário) ou `src/lib.rs` (se for biblioteca).
- `Cargo.toml` descreve a crate (nome, versão, dependências).

Passo 1: Criar o projeto

No terminal:

bash

Copiar código

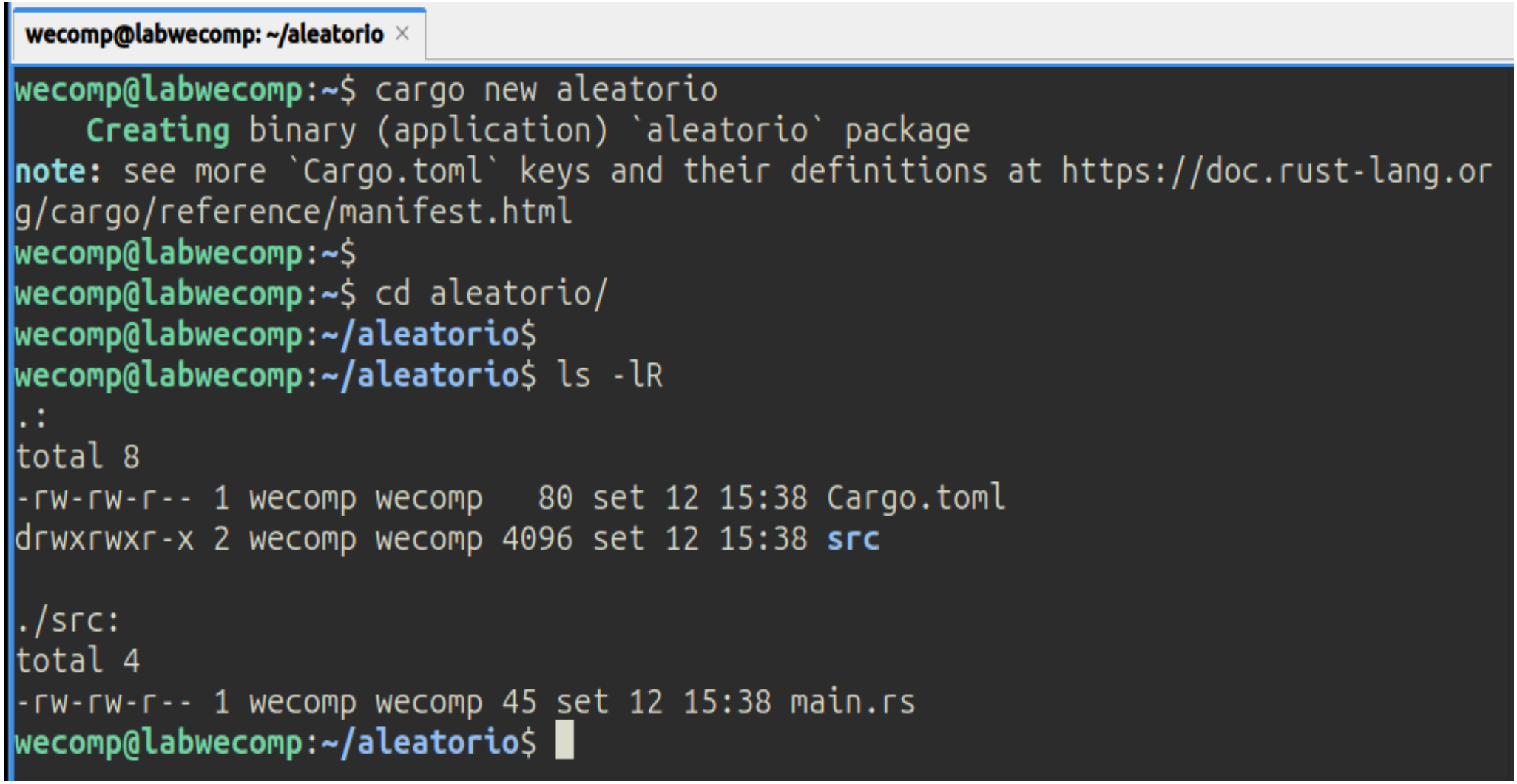
```
cargo new exemplo_cargo
cd exemplo_cargo
```

Isso cria a estrutura:

css

Copiar código

```
exemplo_cargo/
├── Cargo.toml
└── src/
    └── main.rs
```



ATENÇÃO!!!!!
Este projeto já está na máquina virtual.
Se for experimentar, utilize outro nome (p.ex. aleatorio2)
E não esqueça: `/home/wecomp/rust/aleatorio`

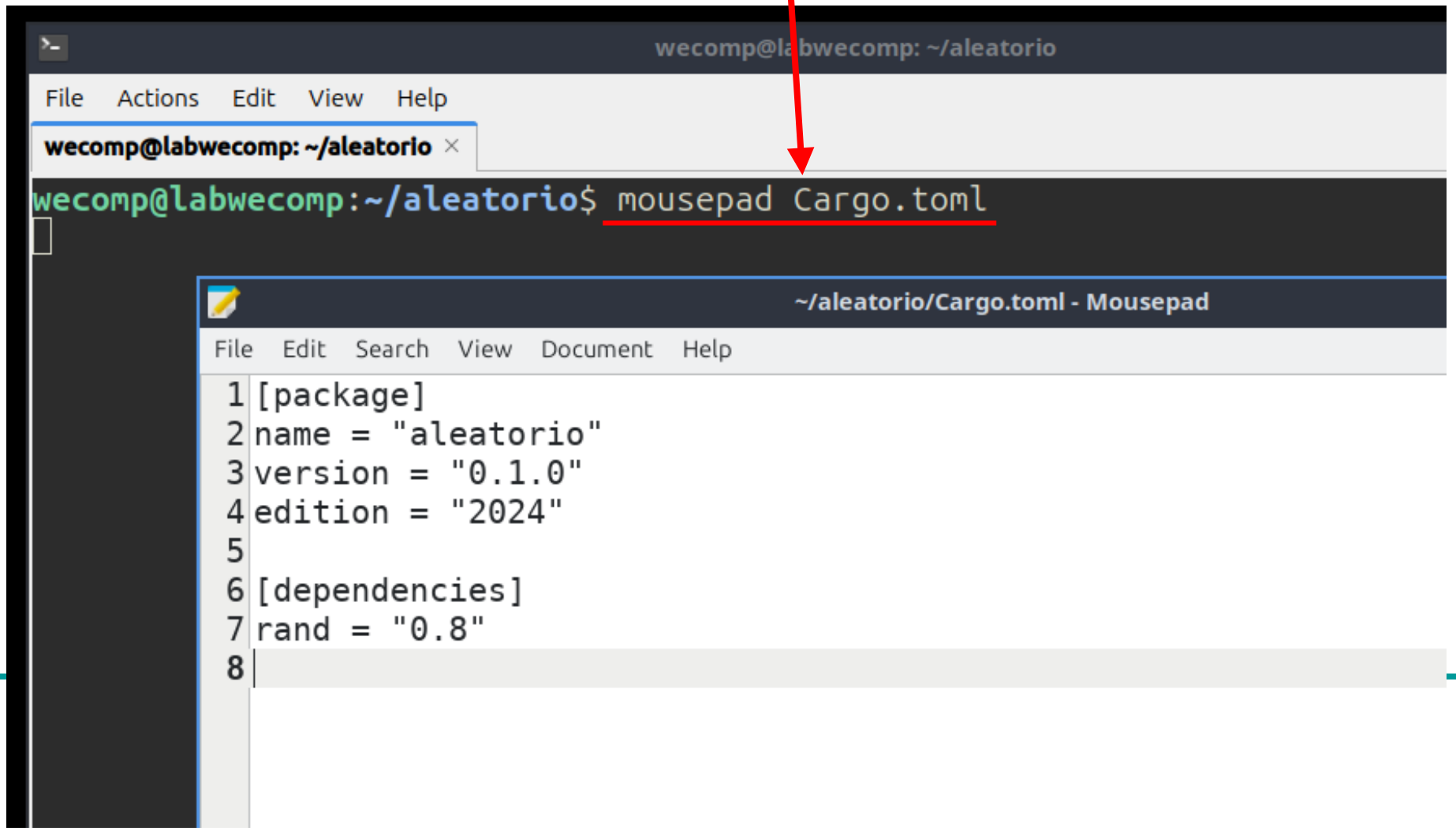
Passo 2: Adicionar dependência

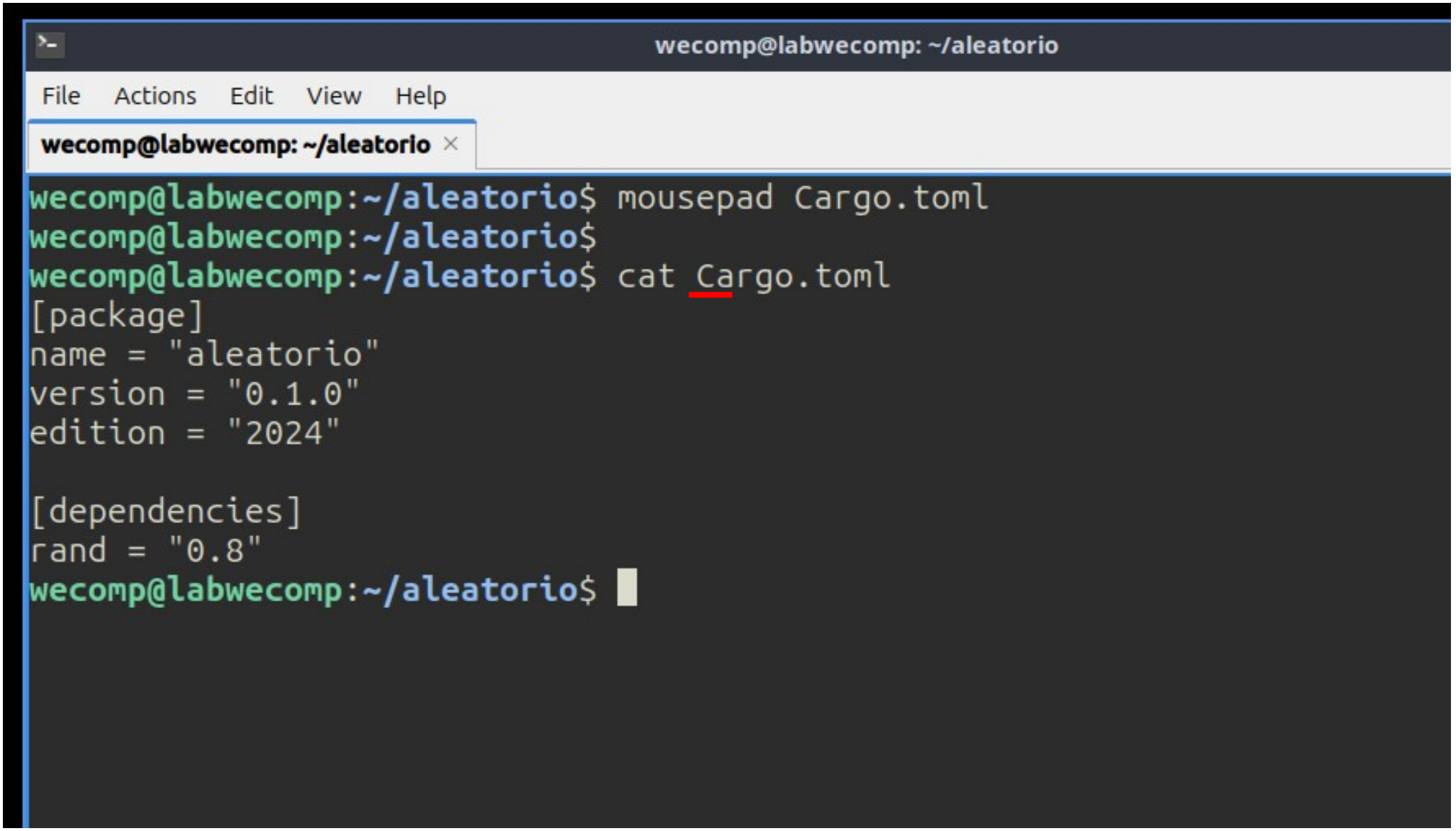
No arquivo `Cargo.toml`, adicione em `[dependencies]`:

toml

Copiar código

```toml
[dependencies]
rand = "0.8"
```




wecomp@labwecomp: ~/aleatorio
File Actions Edit View Help
wecomp@labwecomp: ~/aleatorio
wecomp@labwecomp:~/aleatorio$ mousepad Cargo.toml
wecomp@labwecomp:~/aleatorio$
wecomp@labwecomp:~/aleatorio$ cat Cargo.toml
[package]
name = "aleatorio"
version = "0.1.0"
edition = "2024"
[dependencies]
rand = "0.8"
wecomp@labwecomp:~/aleatorio$

Passo 3: Editar o `main.rs`

Abra `src/main.rs` e troque o conteúdo por:

rust

Copiar código

```rust
use rand::Rng; // traz o trait para gerar números aleatórios

fn main() {
    let mut rng = rand::thread_rng();
    let numero: i32 = rng.gen_range(1..=100);
    println!("Número aleatório gerado: {}", numero);
}
```

Passo 3: Editar o `main.rs`

Abra `src/main.rs` e troque o conteúdo por:

rust — Copiar código

```rust
use rand::Rng; // traz o trait para gerar números aleatórios

fn main() {
    let mut rng = rand::thread_rng();
    let numero: i32 = rng.gen_range(1..=100);
    println!("Número aleatório gerado: {}", numero);
}
```

Uma **trait** em Rust é um conjunto de métodos que podem ser implementados por diferentes tipos — é como um "contrato" que garante que esses tipos terão certos comportamentos.

Algo como a 'interface' do Java

Agora, sobre a linha:

rust — Copiar código

```rust
use rand::Rng; // traz o trait para gerar números aleatórios
```

1. `rand` é a crate que você adicionou como dependência.
2. `Rng` é uma **trait** dentro dessa crate.
   - Ela define métodos relacionados a geração de números aleatórios, como `gen_range`.
3. Ao escrever `use rand::Rng;`, você está **importando a trait para o escopo do seu código**, permitindo que você use os métodos dela sobre objetos que implementam a trait.

Passo 3: Editar o `main.rs`

Abra `src/main.rs` e troque o conteúdo por:

rust — Copiar código

```rust
use rand::Rng; // traz o trait para gerar números aleatórios

fn main() {
    let mut rng = rand::thread_rng();
    let numero: i32 = rng.gen_range(1..=100);
    println!("Número aleatório gerado: {}", numero);
}
```

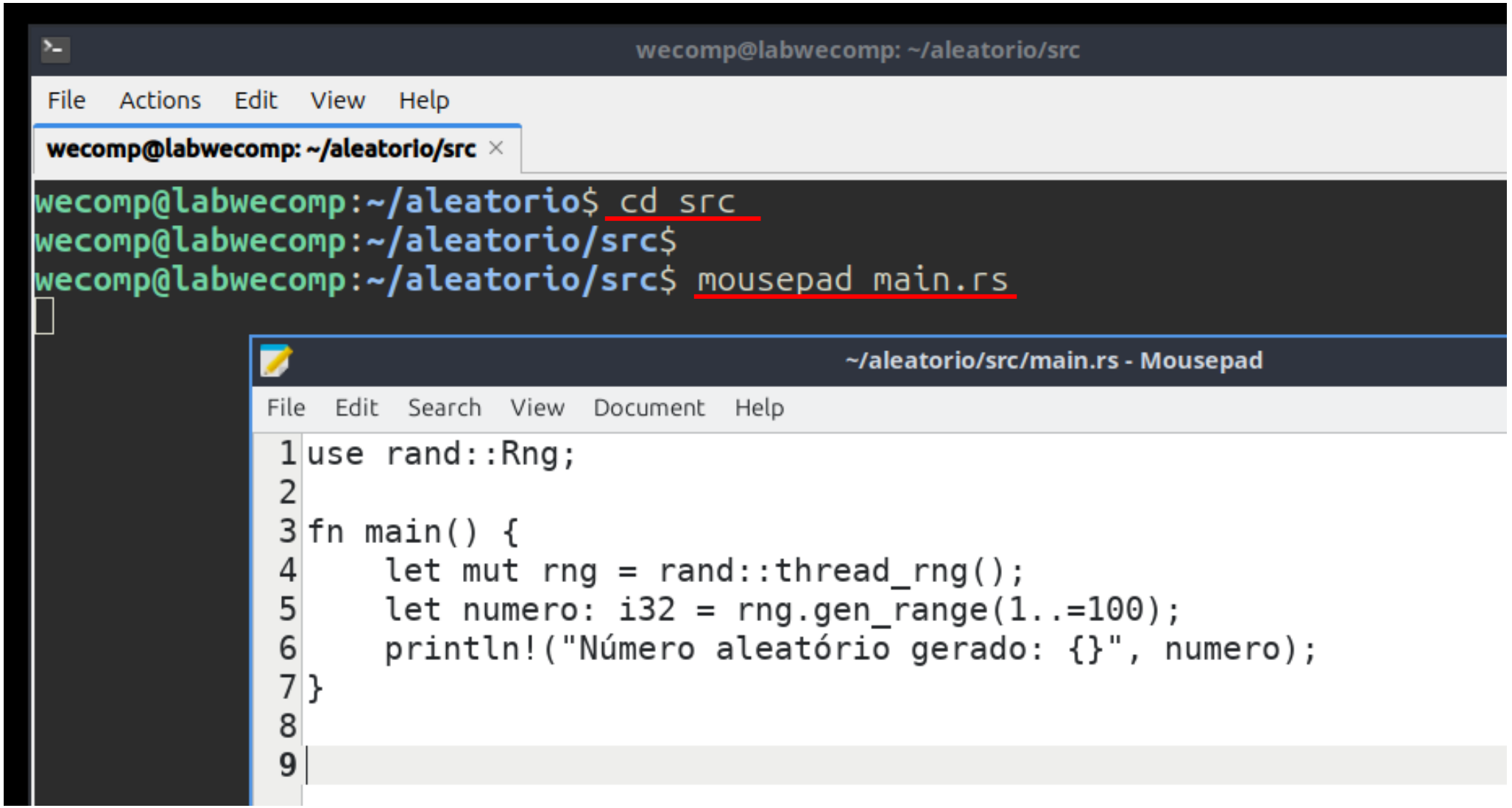
rust — Copiar código

```rust
let mut rng = rand::thread_rng();
let numero: i32 = rng.gen_range(1..=100);
```

- `thread_rng()` retorna um gerador de números aleatórios que **implementa a trait** `Rng`.
- `rng.gen_range(1..=100)` só funciona porque a trait `Rng` foi importada com `use`.

Em resumo:

- **Trait** = conjunto de métodos que tipos podem implementar.
- **use rand::Rng** = permite usar esses métodos no seu código.


wecomp@labwecomp: ~/aleatorio/src
File Actions Edit View Help
wecomp@labwecomp: ~/aleatorio/src
wecomp@labwecomp:~/aleatorio$ cd src
wecomp@labwecomp:~/aleatorio/src$
wecomp@labwecomp:~/aleatorio/src$ mousepad main.rs
~/aleatorio/src/main.rs - Mousepad
File Edit Search View Document Help
1 use rand::Rng;
2
3 fn main() {
4 let mut rng = rand::thread_rng();
5 let numero: i32 = rng.gen_range(1..=100);
6 println!("Número aleatório gerado: {}", numero);
7 }
8
9

Passo 4: Rodar o projeto

```bash
cargo run
```

Estar na pasta do projeto !!!! Não na src !!!

Cargo irá:

1. Baixar a crate `rand` do crates.io.
2. Compilar tanto `rand` quanto o seu código.
3. Executar o programa.

```
wecomp@labwecomp:~/aleatorio$ cd src
wecomp@labwecomp:~/aleatorio/src$
wecomp@labwecomp:~/aleatorio/src$ mousepad main.rs
wecomp@labwecomp:~/aleatorio/src$ #comando para voltar para a pasta anterior
wecomp@labwecomp:~/aleatorio/src$
wecomp@labwecomp:~/aleatorio/src$ cd ..
wecomp@labwecomp:~/aleatorio$
wecomp@labwecomp:~/aleatorio$ cargo run
    Updating crates.io index
     Locking 14 packages to latest Rust 1.89.0 compatible versions
      Adding rand v0.8.5 (available: v0.9.2)
  Downloaded getrandom v0.2.16
  Downloaded rand_chacha v0.3.1
  Downloaded ppv-lite86 v0.2.21
  Downloaded rand_core v0.6.4
  Downloaded cfg-if v1.0.3
  Downloaded rand v0.8.5
  Downloaded zerocopy v0.8.27
  Downloaded libc v0.2.175
  Downloaded 8 crates (1.2MiB) in 0.76s
   Compiling libc v0.2.175
   Compiling zerocopy v0.8.27
   Compiling cfg-if v1.0.3
   Compiling getrandom v0.2.16
   Compiling ppv-lite86 v0.2.21
   Compiling rand_core v0.6.4
   Compiling rand_chacha v0.3.1
   Compiling rand v0.8.5
   Compiling aleatorio v0.1.0 (/home/wecomp/aleatorio)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 6.96s
     Running `target/debug/aleatorio`
Número aleatório gerado: 16
```

```
wecomp@labwecomp:~/aleatorio$
wecomp@labwecomp:~/aleatorio$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.08s
     Running `target/debug/aleatorio`
Número aleatório gerado: 14
wecomp@labwecomp:~/aleatorio$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `target/debug/aleatorio`
Número aleatório gerado: 27
wecomp@labwecomp:~/aleatorio$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `target/debug/aleatorio`
Número aleatório gerado: 71
wecomp@labwecomp:~/aleatorio$
```

Pra fechar...

lendo uma string do teclado.

33.rs

```
1  use std::io;
2
3  fn ler_teclado() -> String {
4
5      let mut entrada = String::new();
6      io::stdin()
7          .read_line(&mut entrada)
8          .expect("Falha ao ler a entrada");
9      entrada.trim().to_string() // remove \n e retorna a string
10
11 }
12
13
14 fn main() {
15
16     println!("Digite algo:");
17     let texto = ler_teclado();
18     println!("Você digitou: {}", texto);
19
20 }
21
```

33.rs

```rust
use std::io;

fn ler_teclado() -> String {

    let mut entrada = String::new();
    io::stdin()
        .read_line(&mut entrada)
        .expect("Falha ao ler a entrada");
    entrada.trim().to_string() // remove \n e retorna a string

}


fn main() {

    println!("Digite algo:");
    let texto = ler_teclado();
    println!("Você digitou: {}", texto);

}
```



```rust
use std::io;
```

- **Importa o módulo `io` da biblioteca padrão (`std`).**
- Esse módulo fornece funções para entrada e saída, incluindo leitura do teclado (`stdin`) e escrita na tela (`stdout`).

33.rs

```rust
use std::io;

fn ler_teclado() -> String {

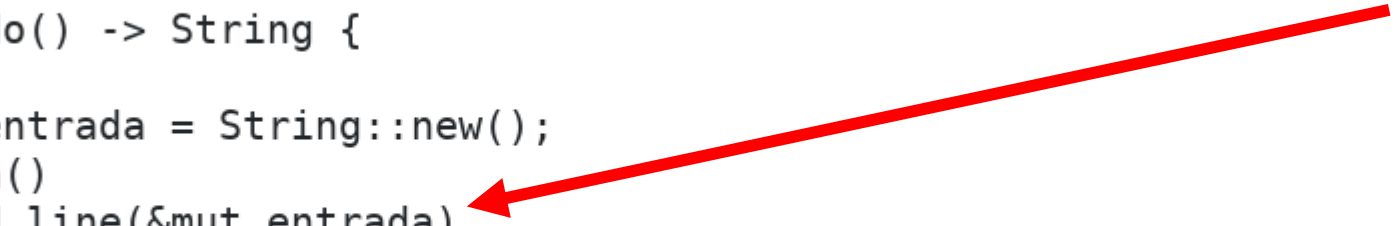
    let mut entrada = String::new();
    io::stdin()
        .read_line(&mut entrada)
        .expect("Falha ao ler a entrada");
    entrada.trim().to_string() // remove \n e retorna a string

}


fn main() {

    println!(
    let texto
    println!(

}
```



```rust
io::stdin()
    .read_line(&mut entrada)
    .expect("Falha ao ler a entrada");
```

- `io::stdin()` retorna o **ponto de entrada padrão**, ou seja, o teclado.
- `.read_line(&mut entrada)` lê uma linha do teclado e coloca o texto dentro da variável `entrada`.
  - O `&mut entrada` passa a variável **como referência mutável**, permitindo que `read_line` escreva dentro dela.
- `.expect("Falha ao ler a entrada")` trata erros:
  - Se ocorrer algum problema ao ler o teclado, o programa para e imprime a mensagem `"Falha ao ler a entrada"`.
  - Isso é importante porque `read_line` retorna um `Result`, que precisa ser tratado.

33.rs

```rust
use std::io;

fn ler_teclado() -> String {

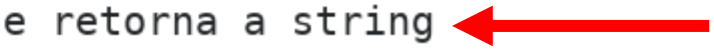
    let mut entrada = String::new();
    io::stdin()
        .read_line(&mut entrada)
        .expect("Falha ao ler a entrada");
    entrada.trim().to_string() // remove \n e retorna a string

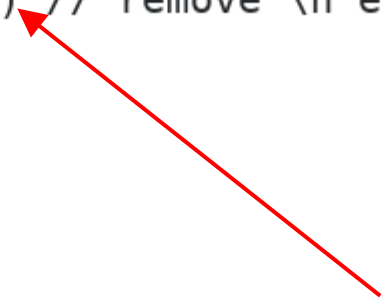
}


fn main() {

    println!("Digite algo:");
    let texto = ler_teclado();
    println!("Você digitou: {}", texto);

}
```





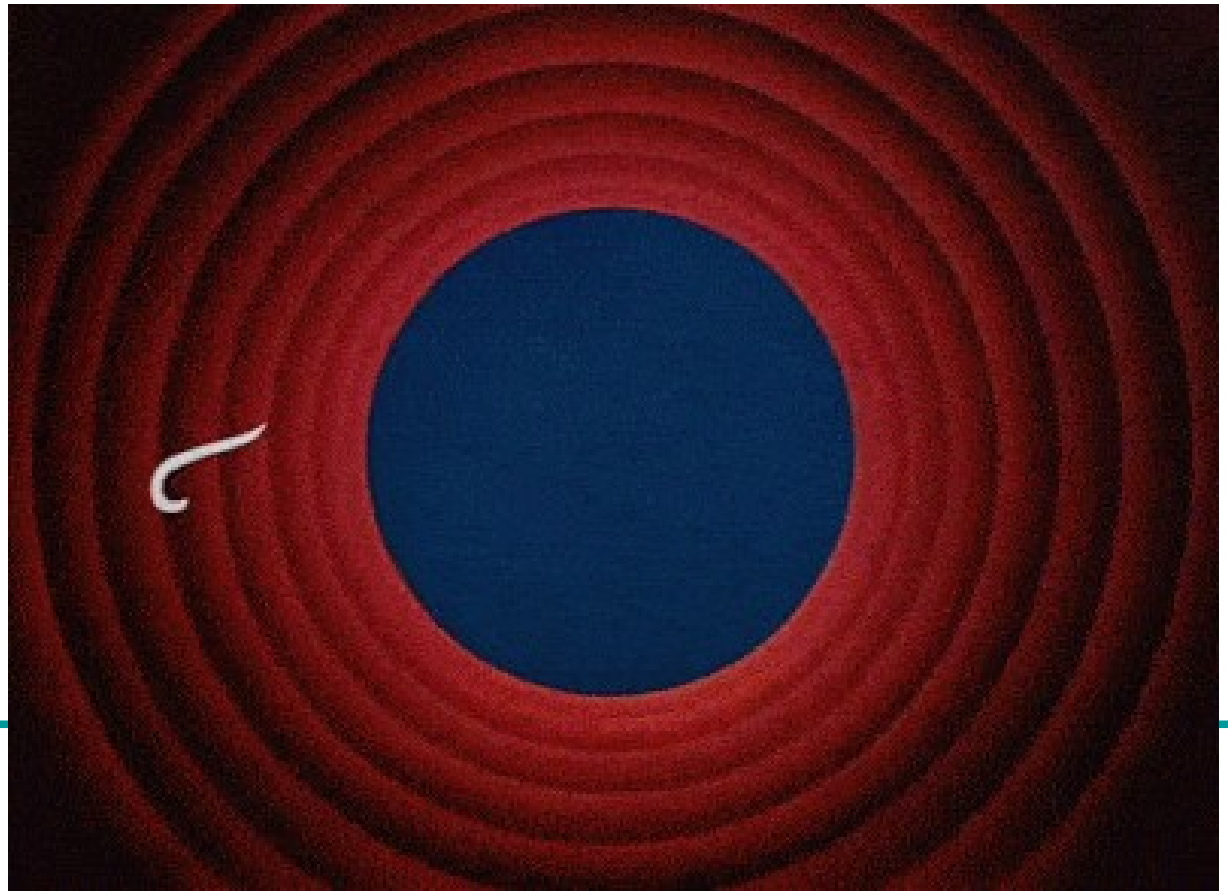
Note, sem ; !!

```rust
entrada.trim().to_string() // remove \n e retorna a string
```

- `entrada.trim()` remove **espaços em branco e quebras de linha** do começo e do fim da string.
- `.to_string()` cria uma nova `String` com o resultado.
- Esse valor é **retornado** pela função `ler_teclado()`.

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 33.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./33
Digite algo:
Obrigado pela atenção!!!!! Espero que tenham gostado!!!!
Você digitou: Obrigado pela atenção!!!!! Espero que tenham gostado!!!!
wecomp@labwecomp:~$
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 33.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./33
Digite algo:
Obrigado pela atenção!!!!! Espero que tenham gostado!!!!
Você digitou: Obrigado pela atenção!!!!! Espero que tenham gostado!!!!
wecomp@labwecomp:~$
```



Foi só a ponta do iceberg....
Bons estudos!!!



Avaliação 1

`Vec`

Um `Vec` (vetor dinâmico) é uma estrutura que permite armazenar vários valores do mesmo tipo. Seu tamanho pode aumentar ou diminuir durante a execução do programa.

Exemplo:

Rust

```rust
let mut nomes = Vec::new();

nomes.push(String::from("Ana"));
nomes.push(String::from("Carlos"));

println!("{}", nomes[0]); // Ana
```

Nesse exemplo, o vetor `nomes` armazena várias cadeias de caracteres (`String`).

`struct`

Uma `struct` (estrutura) permite agrupar diferentes informações relacionadas em um único tipo de dado. Ela é utilizada quando vários atributos representam uma mesma entidade.

Exemplo:

Rust

```rust
struct Livro {
    titulo: String,
    autor: String,
    paginas: u32,
}

let livro = Livro {
    titulo: String::from("O Hobbit"),
    autor: String::from("J. R. R. Tolkien"),
    paginas: 336,
};

println!("{}", livro.titulo);
```

CRUD de livros

- Crie uma struct Livro, com:
  - titulo, autor, editora, núm.páginas
- Armazene os “livros” em um vec
  - Só em memória mesmo
  - Não precisa de índice, utilize o titulo para pesquisar
    - Admita que não haverá repetições!
- Crie funções de crud: inserir, alterar, apagar, exibir um livro, exibir todos os livros
  - No alterar, altere todos os dados (pra simplificar!)
- Crie um menu principal para chamar essas funções.
- Só isso!



```
[/home/carlao2005] > ./crud

==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
1
Título:
2001 Odisséia Espacial
Autor:
Arthur C Clarke
Editora:
Pearson
Número de páginas:
150
Livro cadastrado com sucesso!
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
1
Título:
2010 Odisséia Espacial 2
Autor:
Arthur C Clarke
Editora:
Pearson
Número de páginas:
250
Livro cadastrado com sucesso!
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
5

=== LIVROS CADASTRADOS ===
Título: 2001 Odisséia Espacial | Autor: Arthur C Clarke | Editora: Pearson | Páginas: 150
Título: 2010 Odisséia Espacial 2 | Autor: Arthur C Clarke | Editora: Pearson | Páginas: 250
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
4
Informe o título do livro:
2001 Odisséia Espacial

Título: 2001 Odisséia Espacial
Autor: Arthur C Clarke
Editora: Pearson
Número de páginas: 150
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
4
Informe o título do livro:
O Hobbit
Livro não encontrado.
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
2
Informe o título do livro que deseja alterar:
2001 Odisséia Espacial
Novo título:
2001 A Space Odyssey
Novo autor:
Arthur Clarke
Nova editora:
Fundamento
Novo número de páginas:
300
Livro alterado com sucesso!
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
5

=== LIVROS CADASTRADOS ===
Título: 2001 A Space Odyssey | Autor: Arthur Clarke | Editora: Fundamento | Páginas: 300
Título: 2010 Odisséia Espacial 2 | Autor: Arthur C Clarke | Editora: Pearson | Páginas: 250
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
3
Informe o título do livro que deseja apagar:
2001 A Space Odyssey
Livro removido com sucesso!

==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
5

=== LIVROS CADASTRADOS ===
Título: 2010 Odisséia Espacial 2 | Autor: Arthur C Clarke | Editora: Pearson | Páginas: 250
```

```
==============================
      CRUD DE LIVROS
==============================
1 - Inserir livro
2 - Alterar livro
3 - Apagar livro
4 - Exibir um livro
5 - Exibir todos os livros
0 - Sair
Escolha uma opção:
0
Encerrando o programa...
[/home/carlao2005] >
```

Entrega

- Trabalhos práticos podem ser feitos em dupla (no máximo!)
  - A mesma dupla para a disciplina inteira.

- Prazo:
  - Até as 23:55 do dia 11/agosto (próxima aula, 3a feira).
    - Não será feito controle de presença neste dia.
    - **Porém a presença na aula está vinculada a uma entrega válida do trabalho.**

- Forma de entrega:
  - Envio de email para **`carlao2005@ifsp.edu.br`**
    - Coloque no assunto: **"TTEC AVALIAÇÃO RUST"**
  - Se feito em dupla, **colocar os nomes dos integrantes no email.**
  - Apenas o código fonte, nada demais...

- Vale nota e presença nas 4 aulas deste dia (mínimo nota 8.0)


obrigado
pela atenção!
RENAN PAVINI